



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

The Protocol Stack

2.2 Lower layers: Network, link, physical

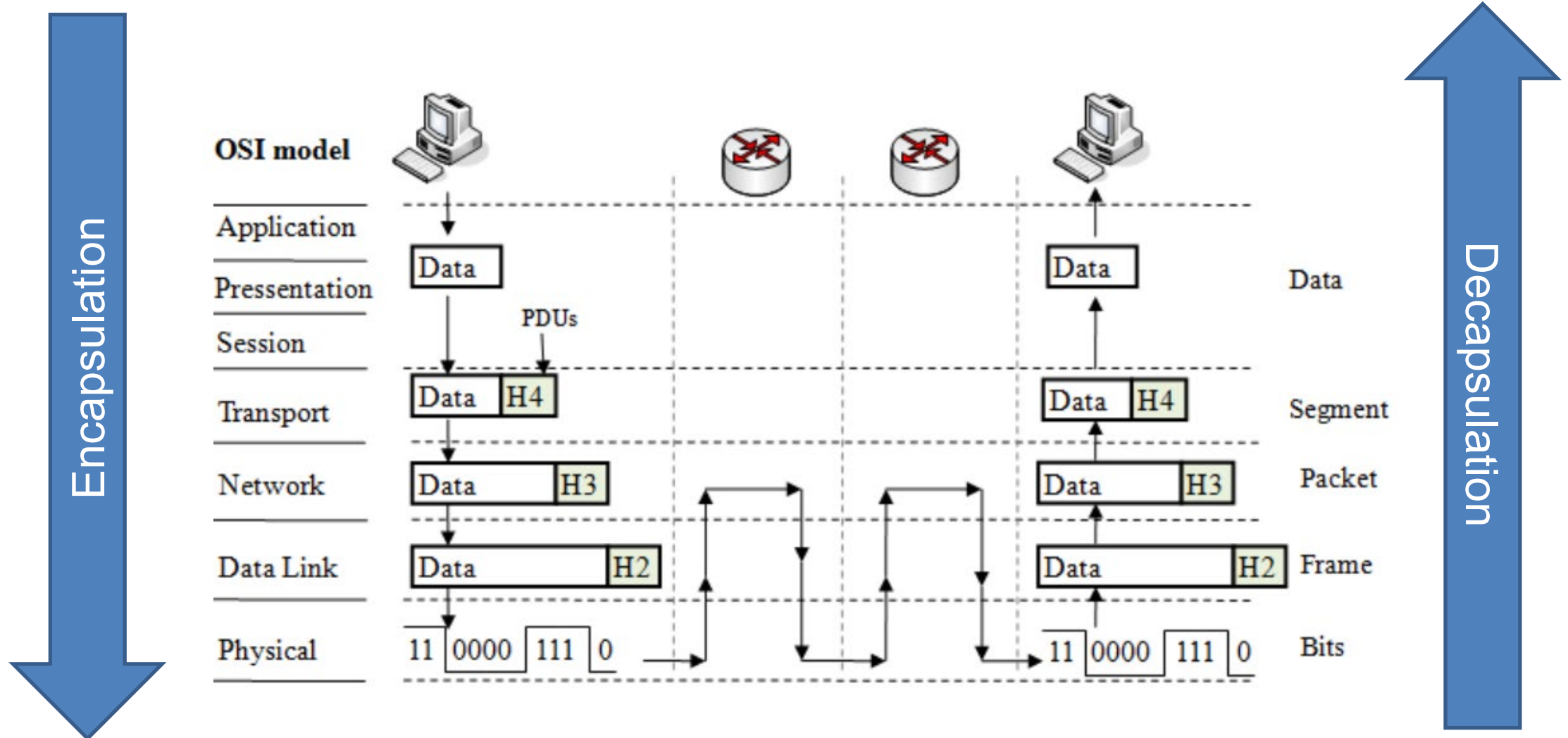
Sergi Abadal (abadal@ac.upc.edu)

1. Introduction
2. The protocol stack (3 sessions)
 - Introduction and upper layers
 - **Lower layers: network, link, physical**
 - Other relevant protocols
 - Putting it all together: Wireshark
3. Impact on programming of on-line games
4. Other stories

Remember from last class

- OSI protocol stack
 - Proposed by the ISO
 - Seven layers with different levels of abstraction
- Encapsulation and decapsulation is the process of a piece of information turning into bits in a wire and viceversa.
- Transport layer:
 - Remote processes can communicate as if they were local
 - TCP adds end-to-end reliability and guarantees over IP
 - UDP does not provide reliability, aimed at speed

Encapsulation (multiple hops)



Transport Layer (sockets)



Sockets

Sockets are endpoints in the communication between two processes in the same or different machines.

They use the same actions as any other I/O functions: Read and write

Two main types for this course: TCP and UDP

Need an Address and a Port

From the lab sessions

Sockets

To create a UDP Socket:

```
newSocket = new Socket(AddressFamily.InterNetwork,  
                        SocketType.Dgram,  
                        ProtocolType.Udp);
```

To create a TCP Socket:

```
newSocket = new Socket(AddressFamily.InterNetwork,  
                        SocketType.Stream,  
                        ProtocolType.Tcp);
```

InterNetwork -> IPv4 family of addresses
InterNetworkV6 -> IPv6 family of addresses

Dgram vs. Stream -> independent packets (Datagrams) vs continuous stream of data

Dgram always for UDP.
Stream always for TCP.

newSocket.Shutdown() -> shutdown the transfer of data, but keep socket alive

newSocket.Close() -> Stop transfers and **destroy** socket.



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

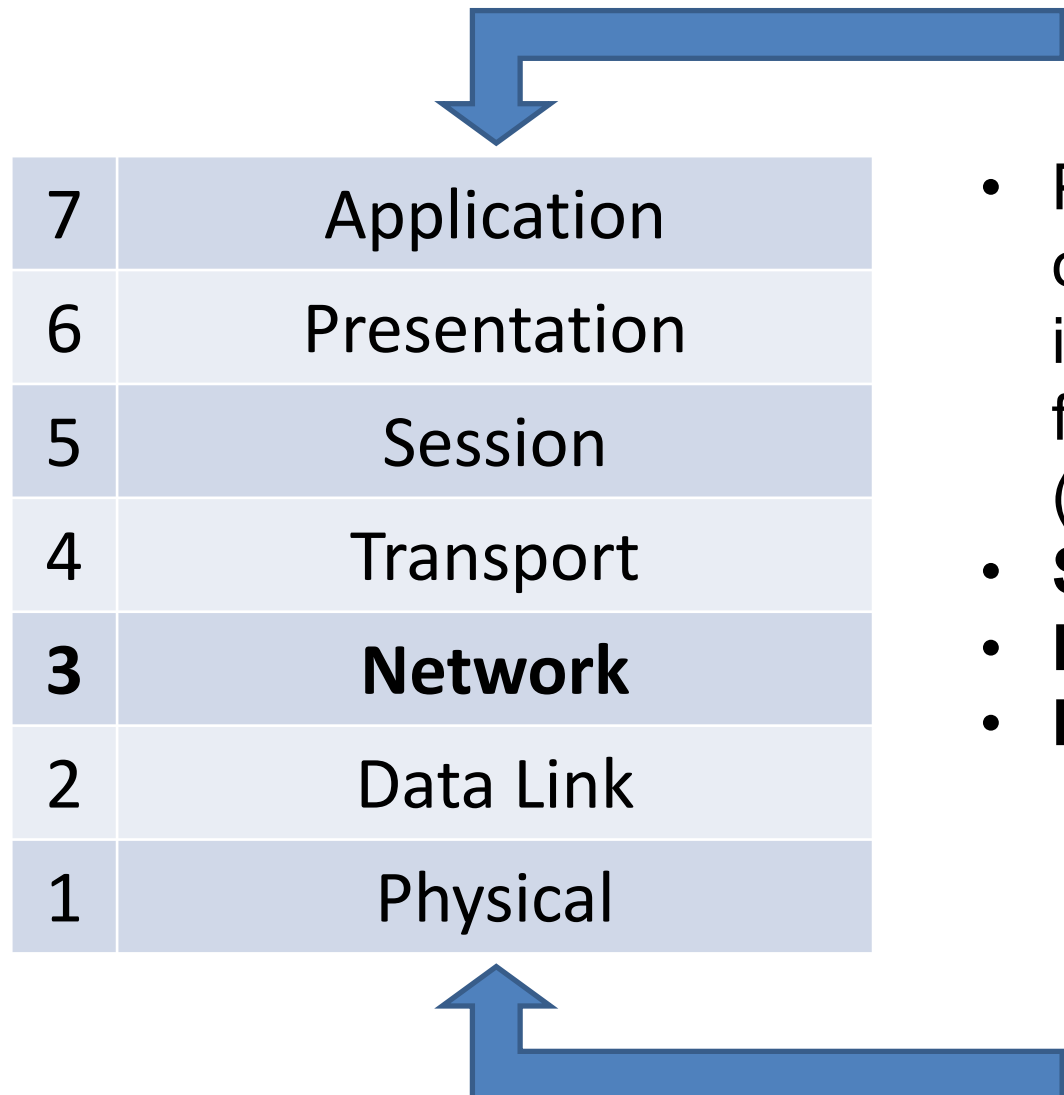
Centre de la Imatge i la Tecnologia Multimèdia

The Network Layer

Internet Protocol (IP)

Network Layer

The game



7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Protocols for routing packets through a set of nodes, from source to final destination. It includes methods for addressing and forwarding, as well as congestion control (change routes).
- **Scope:** two routers within the network
- **Data unit:** packet
- **Example:** IP

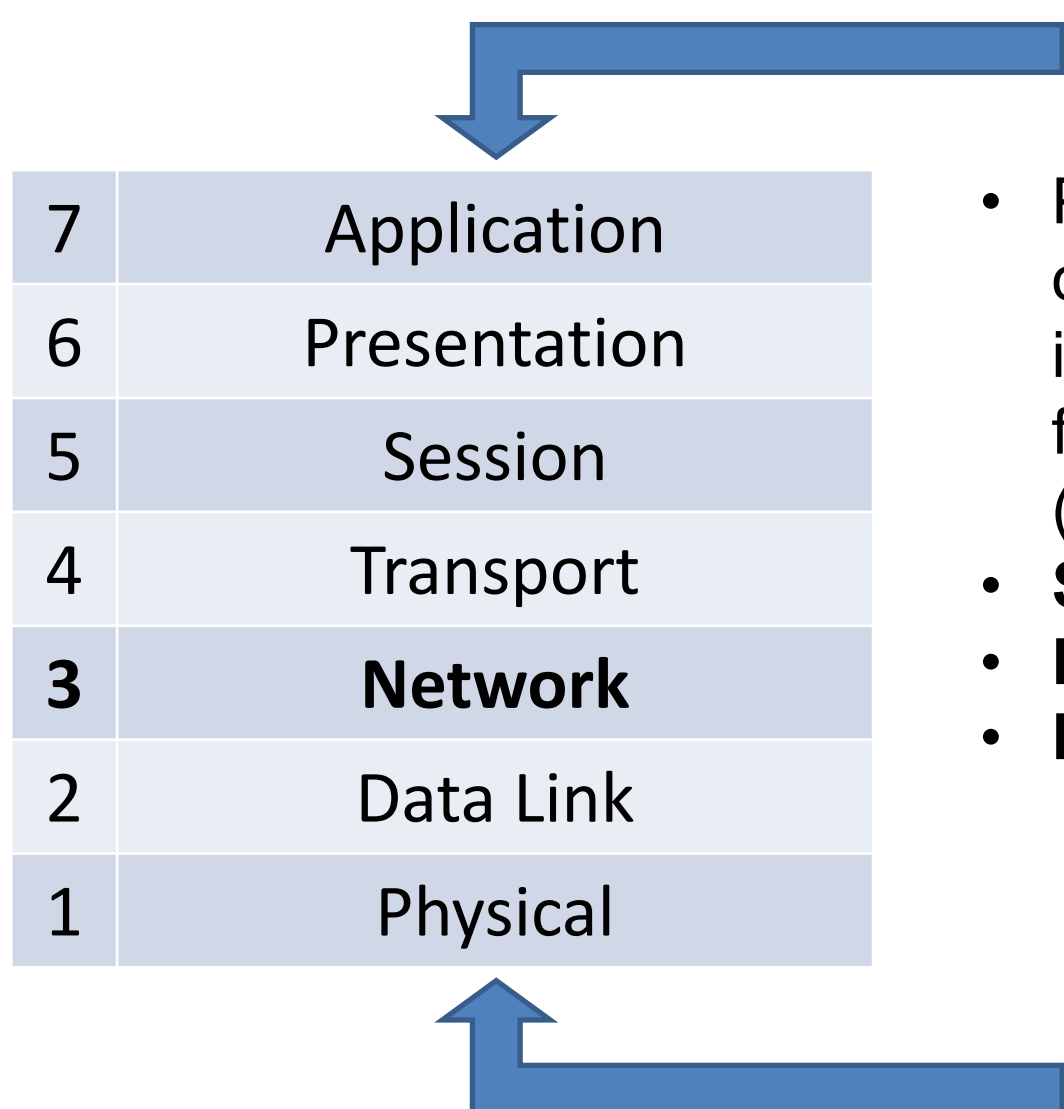
0	4	8	16	19	31
Version	IHL	Type of Service	Total Length		
Identification			Flags	Fragment Offset	
Time To Live		Protocol	Header Checksum		
Source IP Address					
Destination IP Address					
Options					Padding

The wire

- The widely IP protocol takes care of addressing and routing through the network. Characteristics:
 - Aimed at low complexity – routers are simple
 - Best effort – no performance guarantees
 - Each packet is treated independently
 - ordering is not enforced
 - Reliability is generally given by upper layers

Network Layer

The game



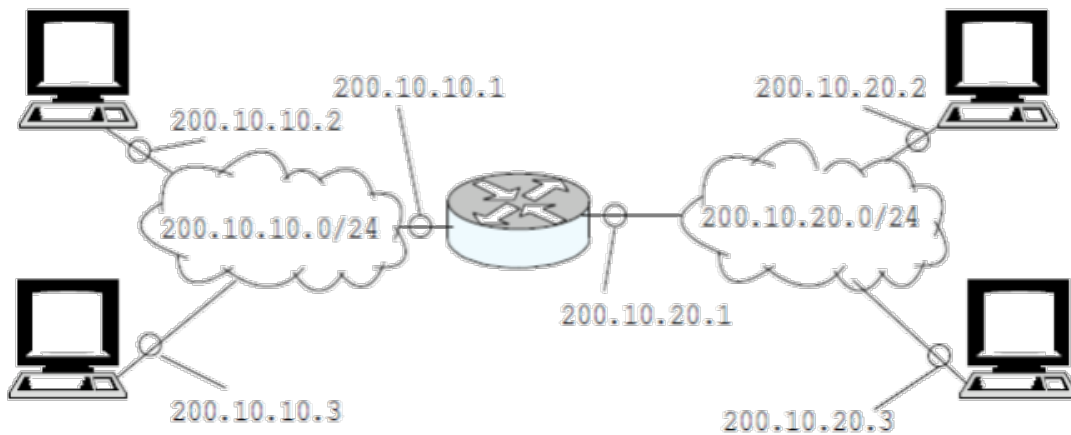
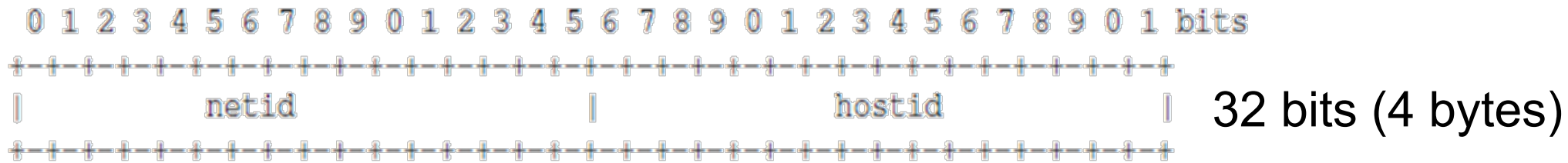
7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Protocols for routing packets through a set of nodes, from source to final destination. It includes methods for **addressing** and forwarding, as well as congestion control (change routes).
- **Scope:** two routers within the network
- **Data unit:** packet
- **Example:** IP

0	4	8	16	19	31
Version	IHL	Type of Service	Total Length		
Identification			Flags	Fragment Offset	
Time To Live		Protocol	Header Checksum		
Source IP Address					
Destination IP Address					
Options					Padding

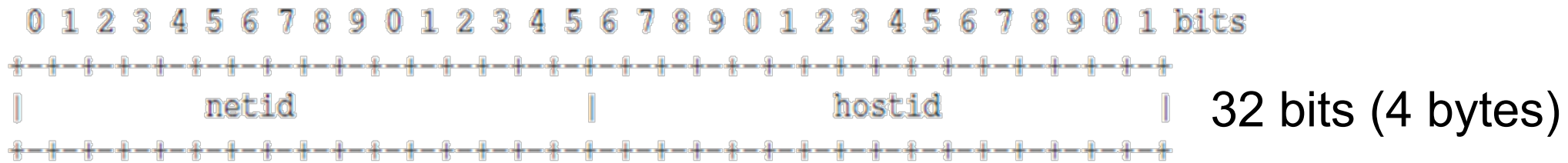
The wire

Internet Protocol: IPv4 addresses



- All the interfaces in the same network have the same **network prefix (netid)**
- The router must assign an address to each of its interfaces
- Each address needs to have the netid of the network where it is connected.
- The special addresses cannot be used to identify the nodes

Internet Protocol: IPv4 addresses



Example:

IP: 95.19.246.51

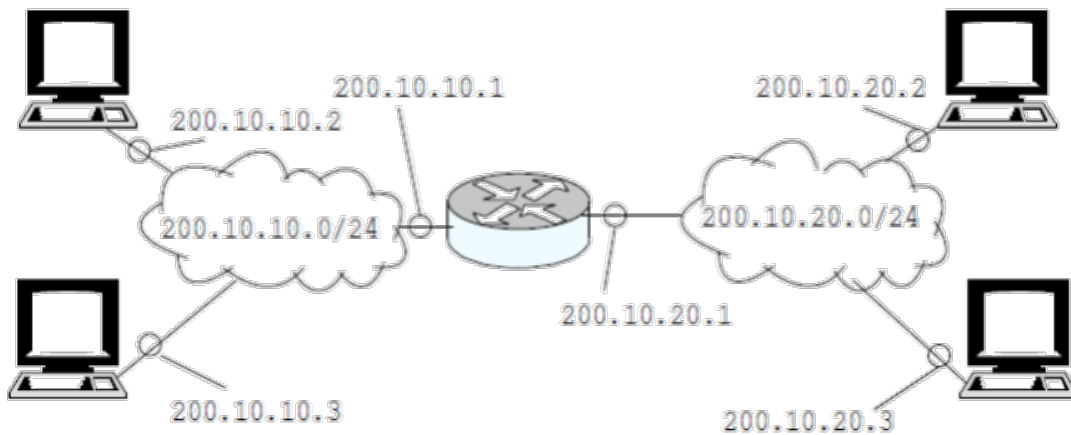
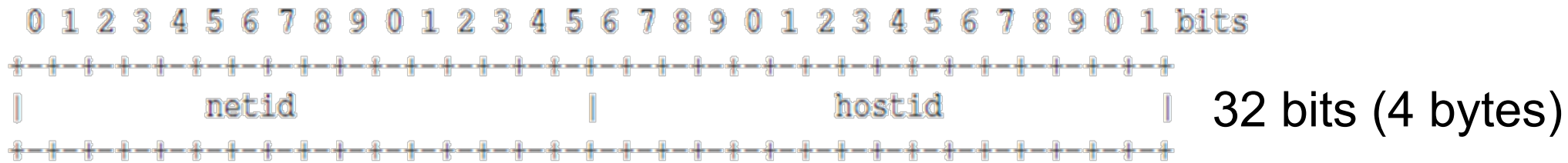
Mask: 255.0.0.0 or /8

NetId = IP AND Mask = 95.0.0.0

HostId = IP NAND Mask = 0.19.246.51

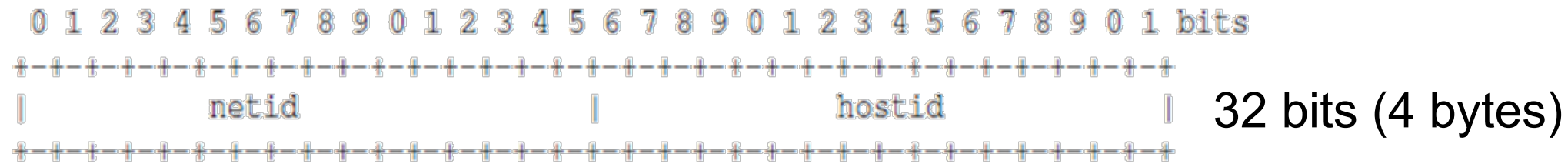
Addresses generally come with a “mask” that decides the division of bits between netid and hostid.

Internet Protocol: IPv4 addresses



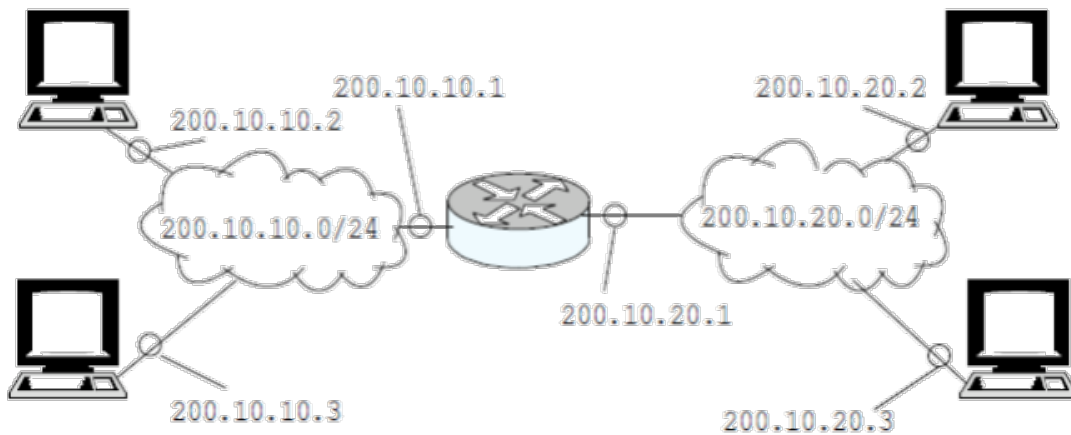
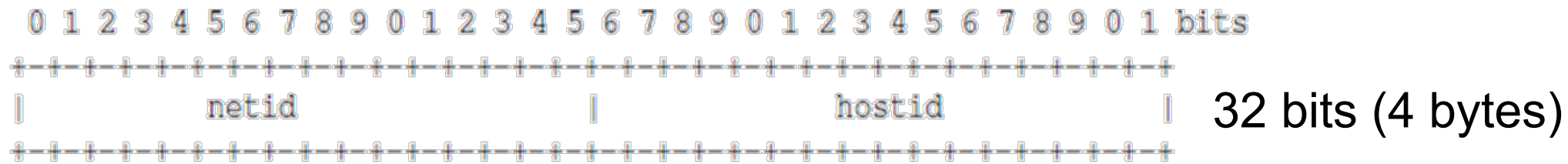
- All the interfaces in the same network have the same network prefix (netid)
- The router must assign an address to each of its interfaces
- Each address needs to have the netid of the network where it is connected.
- **The special addresses cannot be used to identify the nodes**

Internet Protocol: Special addresses



netid	hostid	Meaning
XXX	all 0s	Identifies a network. Used in routing tables.
XXX	all 1s	Broadcast in network XXX
all 0s	all 0s	Identifies “this host” in “this network”. It is used as a source address in auto-configuration protocols (like DHCP)
all 1s	all 1s	Broadcast in “this network”. It is used as a destination address in auto-configuration protocols (like DHCP)
127	XXX	Loopback: communication between processes of the same host with TCP/IP

Internet Protocol: Private IP addresses

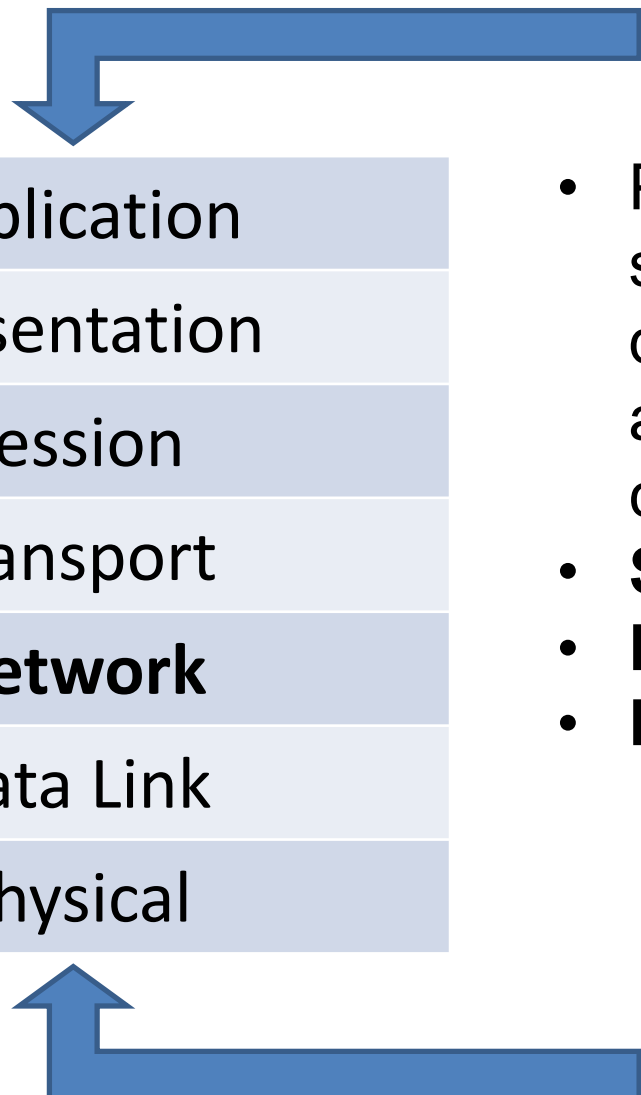


- Networks (or hosts) that use TCP/IP in private networks without a public address make use of the following blocks of reserved addresses (non-routable in Internet)
 - 10.0.0.0 - 10.255.255.255
 - 172.16.0.0 - 172.31.255.255
 - 192.168.0.0 - 192.168.255.255

- All addresses (4000M) have been used as of Feb. 2011
 - Many more are needed (IoT, connected cars...)
- Solution? Put more bits in the address
- This is what it is done by IPv6
 - 128 bits – virtually unlimited number of addresses
- Migration to IPv6 is still underway
- Want to know more? IPv6 guide from ISOC-CAT
<http://www.ipv6peratothom.cat/>

Network Layer

The game



7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Protocols for routing packets through a set of nodes, from source to final destination. It includes methods for addressing and forwarding, as well as congestion control (change routes).
- **Scope:** two routers within the network
- **Data unit:** packet
- **Example:** IP

0	4	8	16	19	31
Version	IHL	Type of Service	Total Length		
Identification			Flags	Fragment Offset	
Time To Live		Protocol	Header Checksum		
Source IP Address					
Destination IP Address					
Options					Padding

The wire

Internet Protocol: Routing Protocol

*Given a destination IP address, **D**, and netId, **N**:*

***if** (**N** matches a directly connected network address netId) *// last hop**

*Deliver datagram to **D** over that network link;*

***else if** (The routing table contains a route for **N**) *// next hop**

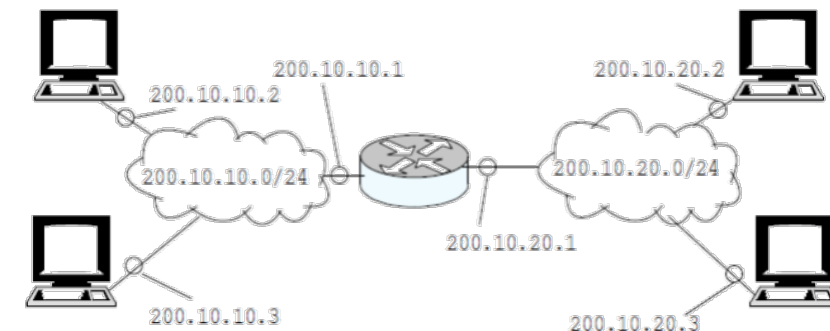
Send datagram to the next-hop address listed in the routing table;

***else if** (a default route exists) *// next hop by default**

Send datagram to the default route;

***else** *// unreachable**

Send a forwarding error message to the originator;



Internet Protocol: Routing Protocol

Given a destination IP address, **D**, and network prefix **N**:

if (**N** matches a directly connected

Deliver datagram to **D** over that

else if (The routing table contains

Send datagram to the next-hop address listed in the routing table,

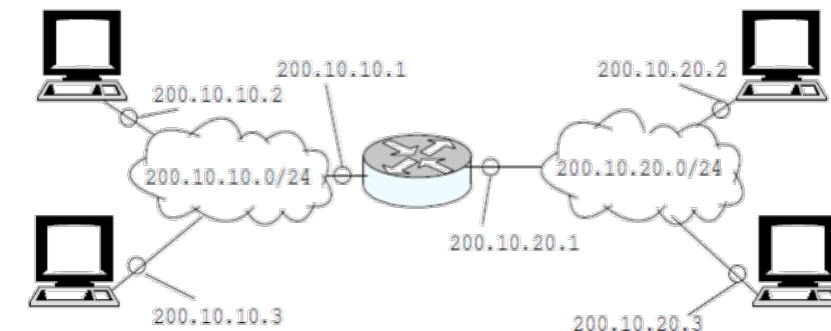
else if (a default route exists) // next hop by default

Send datagram to the default route;

else // unreachable

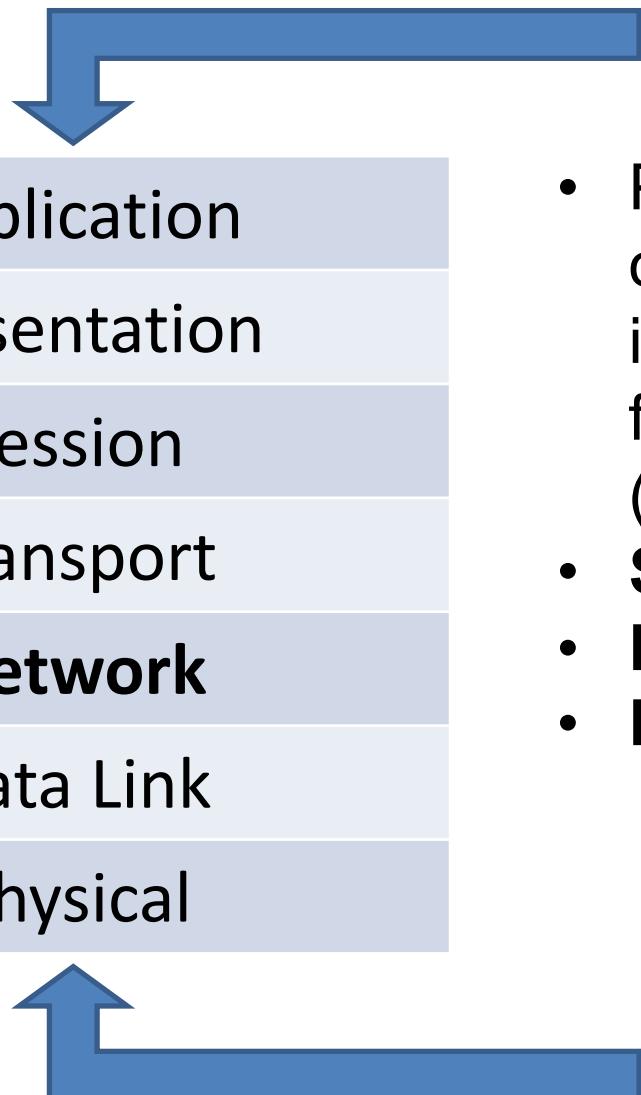
Send a forwarding error message to the originator;

Windows: route print
Unix: netstat -rn



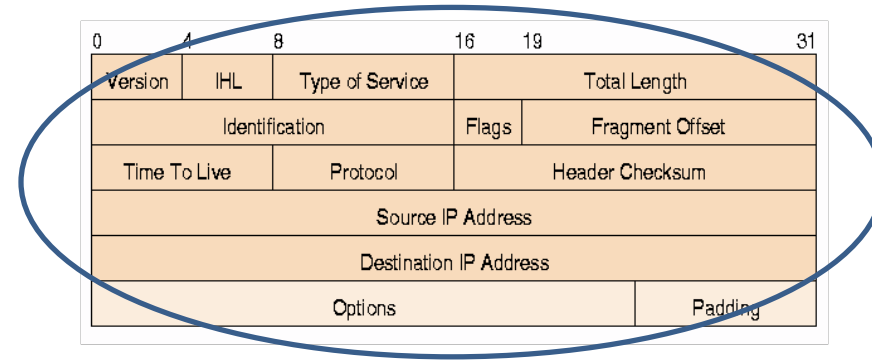
Network Layer

The game



7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

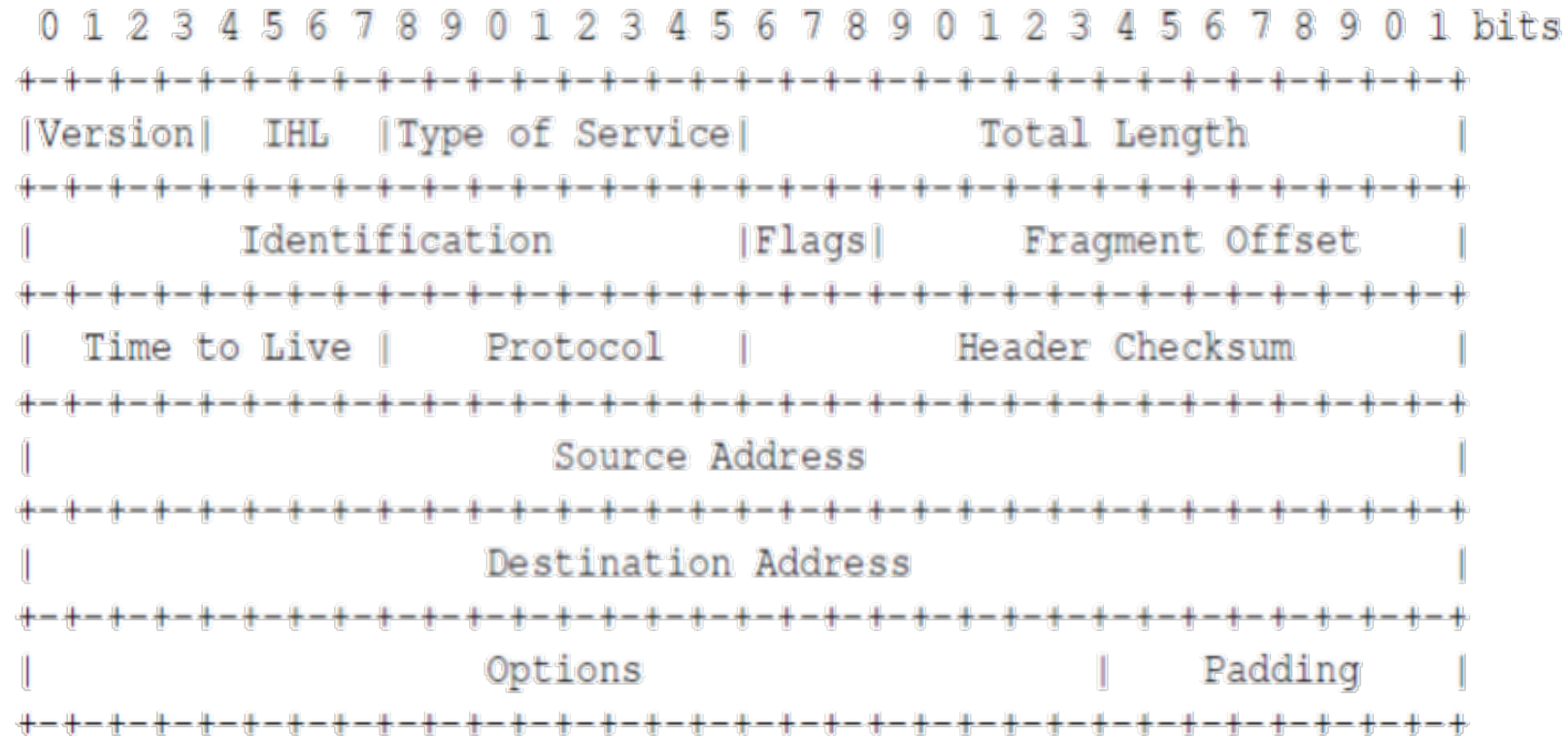
- Protocols for routing packets through a set of nodes, from source to final destination. It includes methods for addressing and forwarding, as well as congestion control (change routes).
- **Scope:** two routers within the network
- **Data unit:** packet
- **Example:** IP



0	4	8	16	19	31
Version	IHL	Type of Service	Total Length		
Identification			Flags	Fragment Offset	
Time To Live		Protocol	Header Checksum		
Source IP Address					
Destination IP Address					
Options					Padding

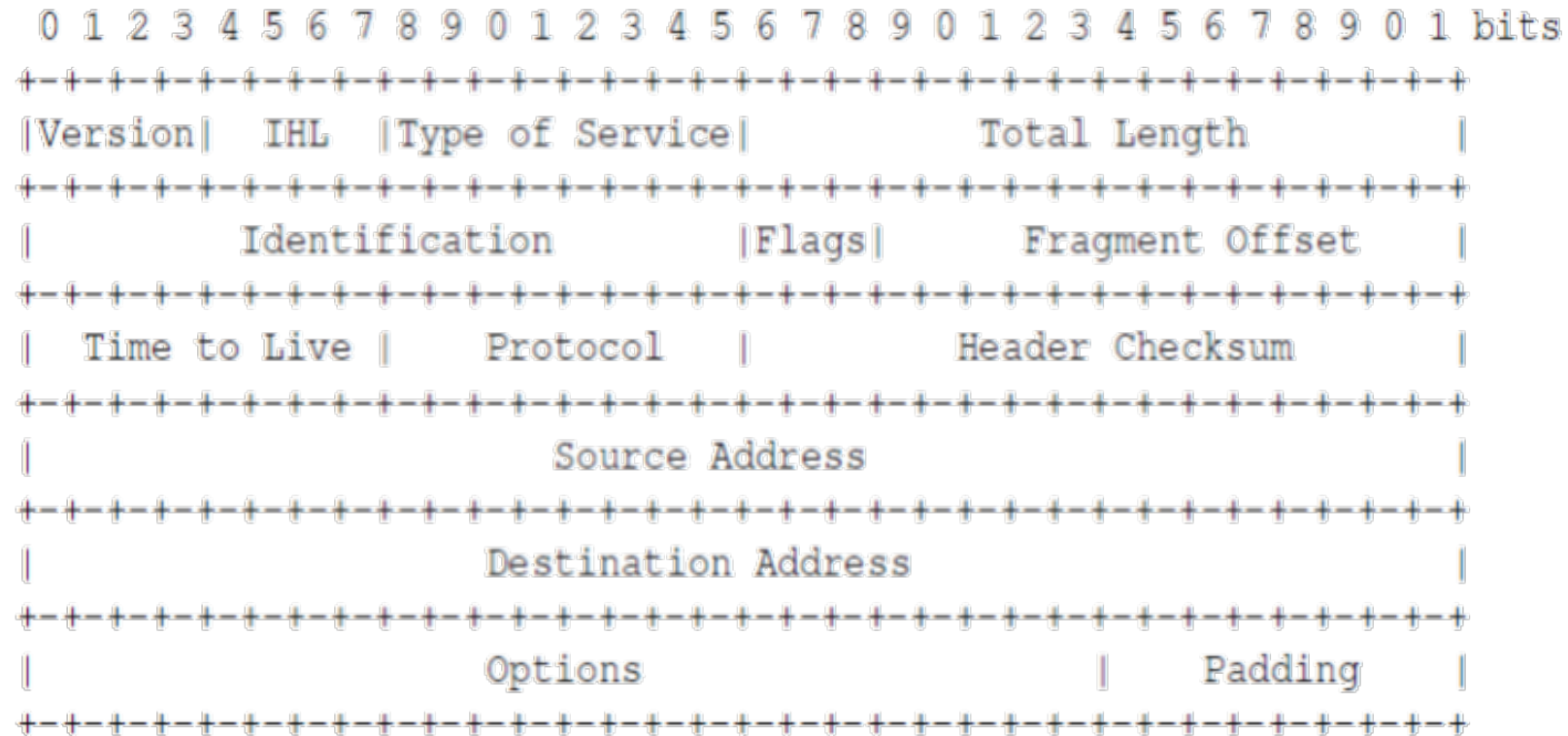
The wire

Internet Protocol: Packet header



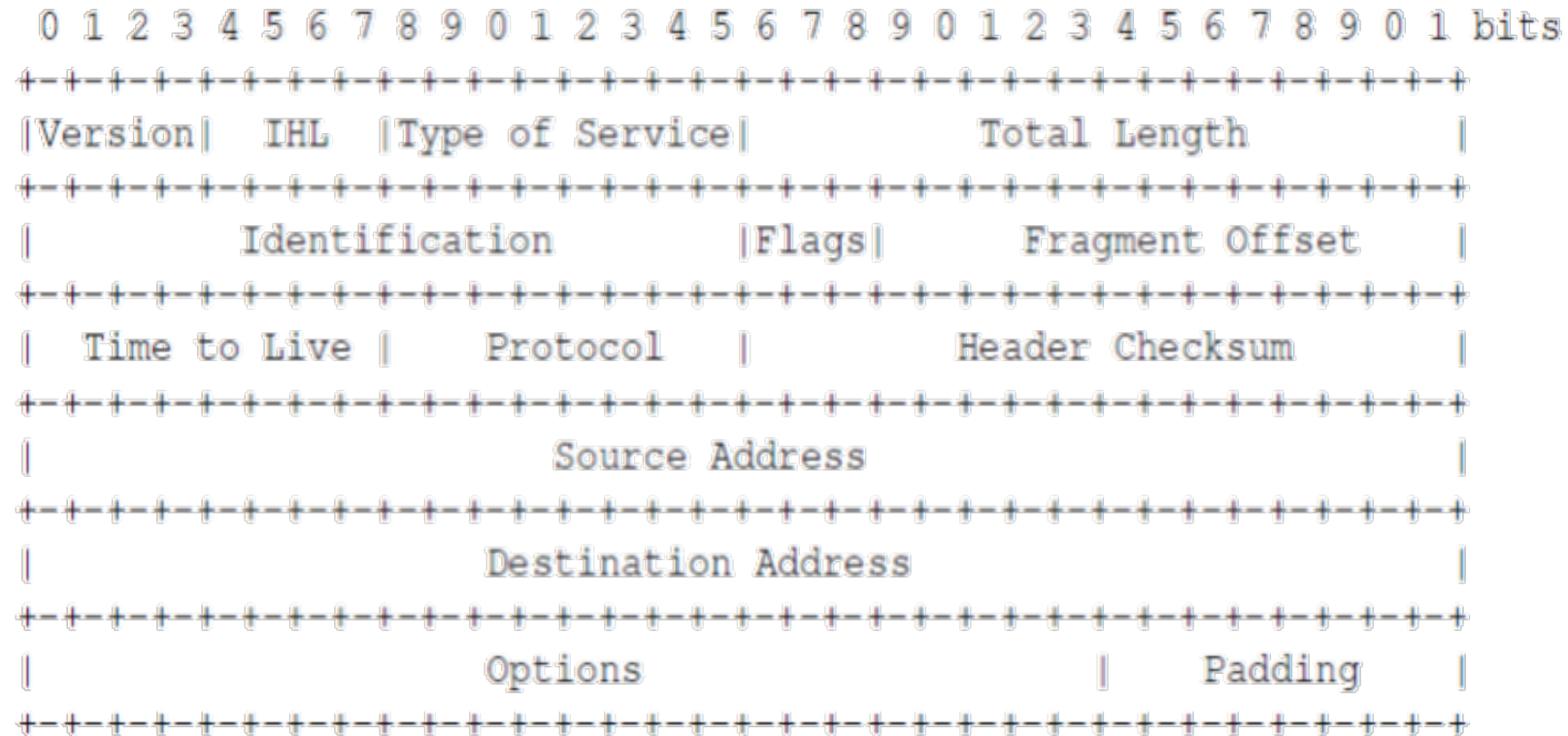
- **Version:** protocol version (i.e. 4 or 6)
- **IHL: IP Header Length**, header length in 32-bit words (without options, the IHL is 5, which is $5 \times 32 / 8 = 20$ bytes)

Internet Protocol: Packet header



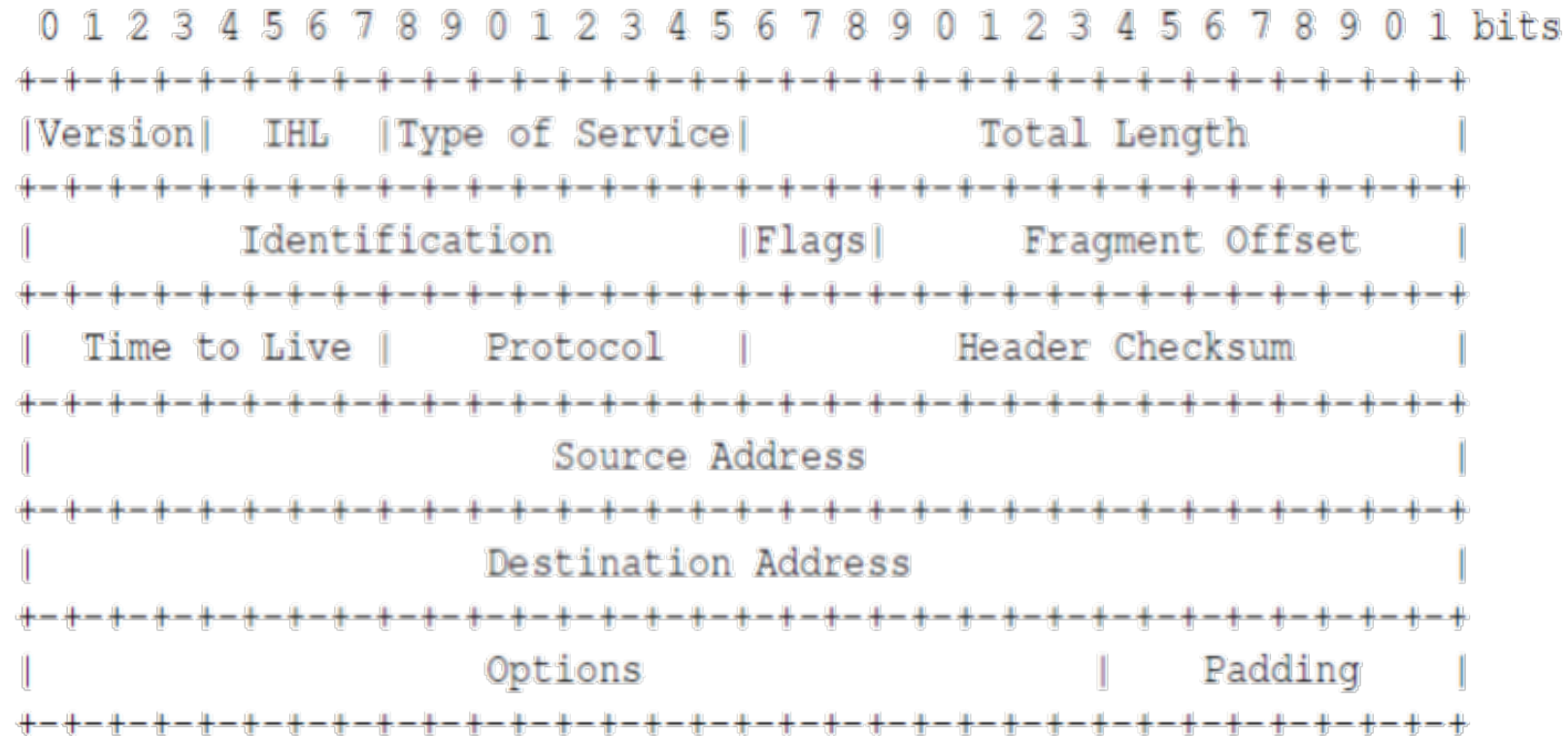
- **Total Length:** length of the datagram in bytes.
- **Identification, Flags i Fragment Offset:** used for fragmentation
- **Time to Live (TTL):** hops remaining before discarding

Internet Protocol: Packet header



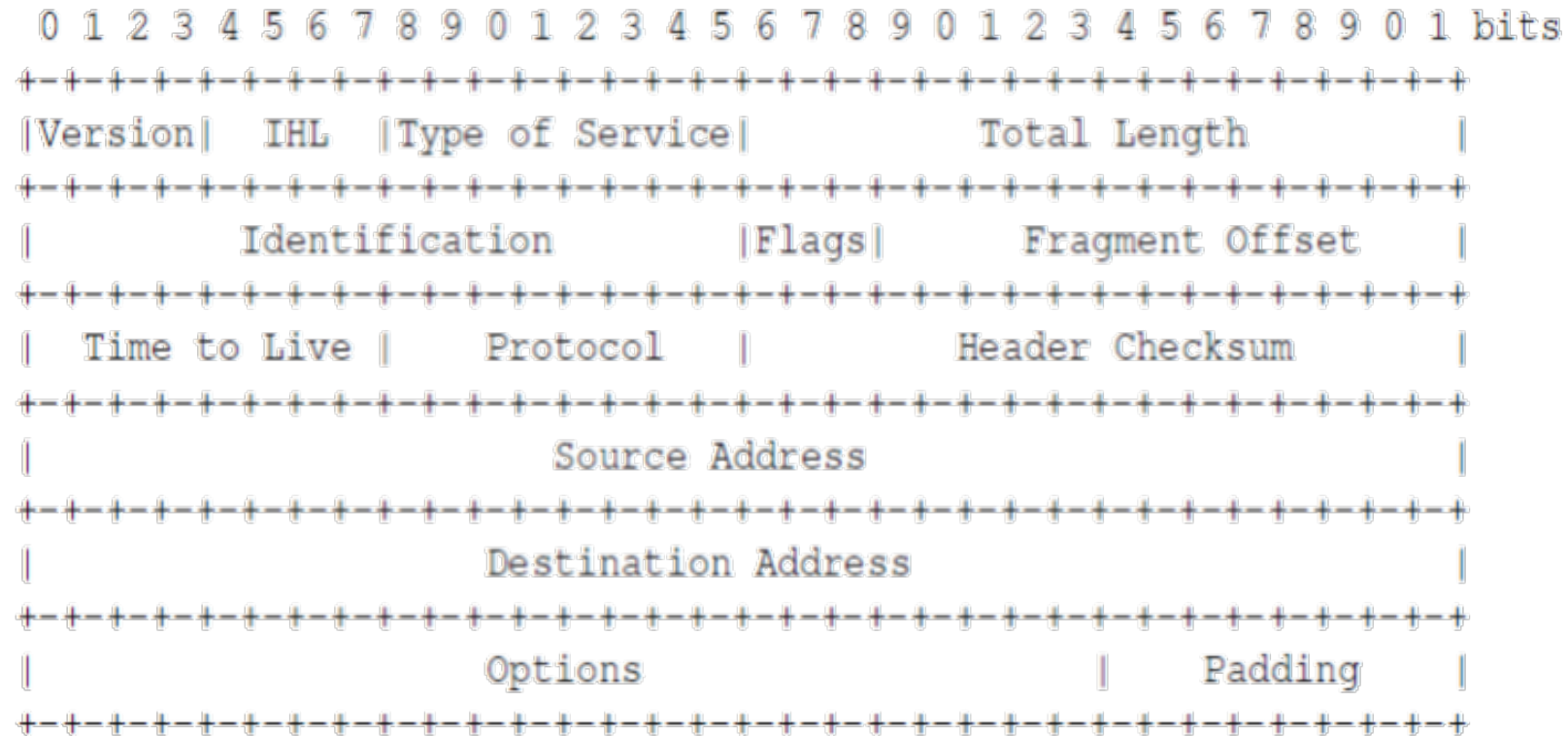
- **Header checksum:** custom checksum of the header to add some reliability at the network level

Internet Protocol: Packet header



- **Source address:** self-described
- **Destination address:** self-described

Internet Protocol: Packet header

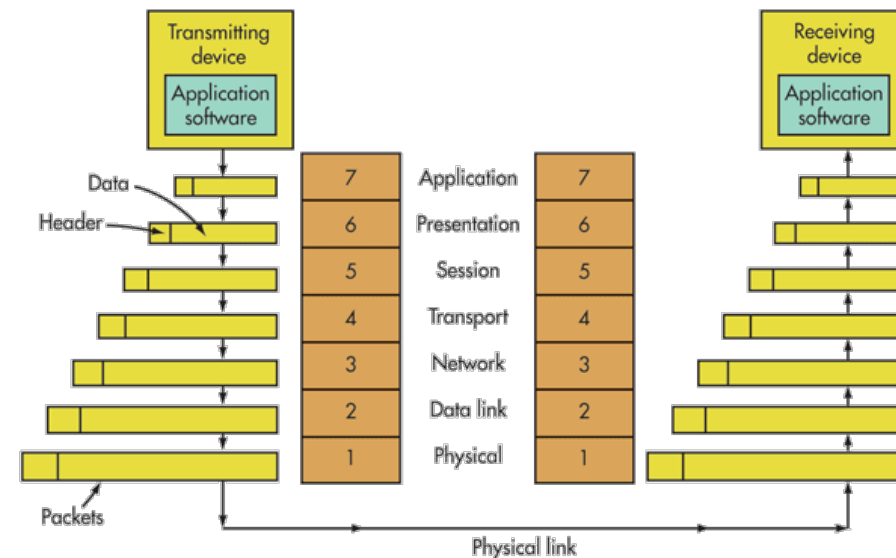


- **Options:** extra bits for some extra functionalities
- **Padding:** bits stuffed to make sure that the datagram is a multiple of 32-bit words

- In networks, there is a definition of **Maximum Transmission Unit (MTU)**: maximum size of a data unit that can go through the links of a network.
- There is a tradeoff between latency/reliability (small MTU) and throughput (large MTU).

Exercise – Maximum data at MTU

- The MTU of the Ethernet protocol (data link layer) is 1500 bytes. This does not include the Ethernet header of 18 bytes.
- Assume that the physical layer does not add any header, while the network layer add 20 bytes, transport adds 20 bytes, and session, presentation and application add 10 bytes each.
- What is the maximum amount of application data sent in a frame?
- What is the minimum ratio of header size divided by amount of application data sent in each frame?

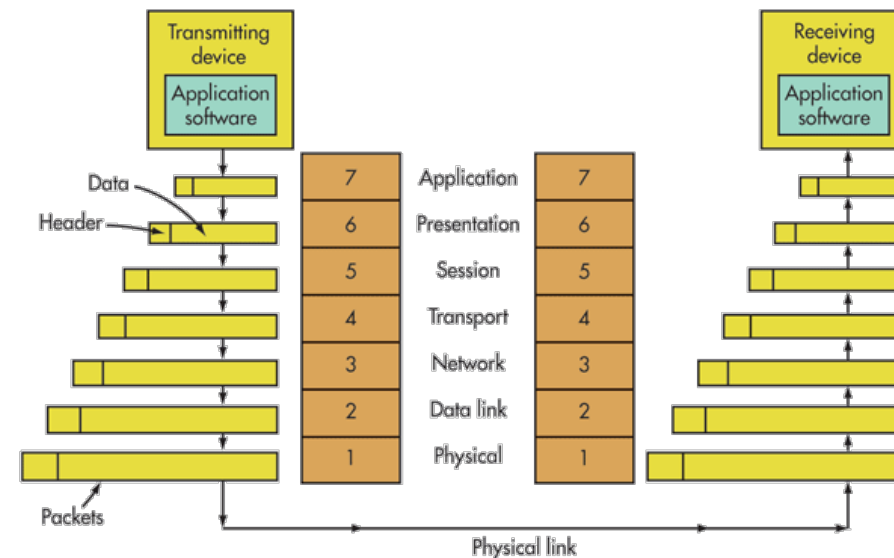


Exercise – Maximum data at MTU

- The maximum amount of application data is:
$$\text{AppData} = 1500 - 20 - 20 - 10 \times 3 = 1430 \text{ Bytes}$$
- The total amount of header:
$$\text{Header} = 18 + 20 + 20 + 10 \times 3 = 88 \text{ Bytes}$$
- Minimum ratio:

$$\text{Ratio} = \text{Header} / \text{AppData} = 88 / 1430 = 6.15\%$$

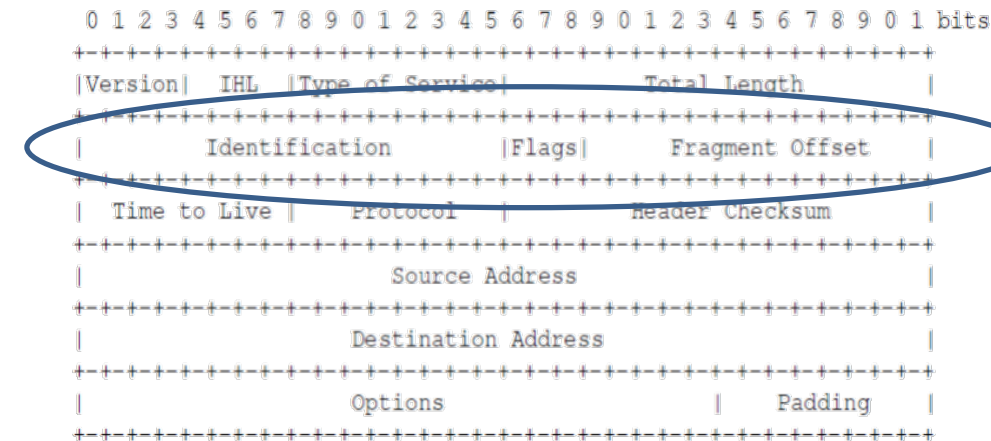
Much better than the example from last class:
Ratio: $628 / 320 = 1.9625 = 196.25\%$



- In networks, there is a definition of **Maximum Transmission Unit (MTU)**: maximum size of a data unit that can go through the links of a network.
- There is a tradeoff between latency/reliability (small MTU) and throughput (large MTU).
- It is responsibility of the IP protocol to fragment packets that exceed the MTU.

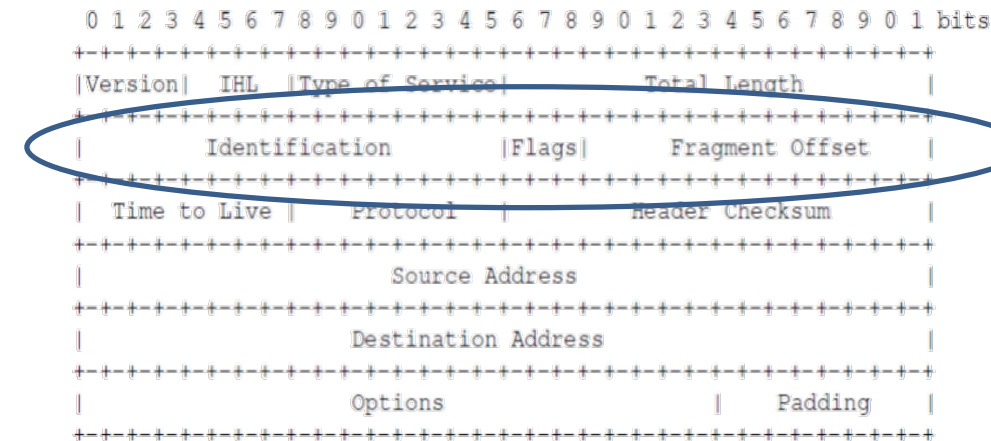
Internet Protocol: Fragmentation

- In networks, there is a definition of **Maximum Transmission Unit (MTU)**: maximum size of a data unit that can go through the links of a network.
- There is a tradeoff between latency/reliability (small MTU) and throughput (large MTU).
- It is responsibility of the IP protocol to fragment packets that exceed the MTU. It uses:
 - Identification
 - Flags
 - Fragment offset



Internet Protocol: Fragmentation

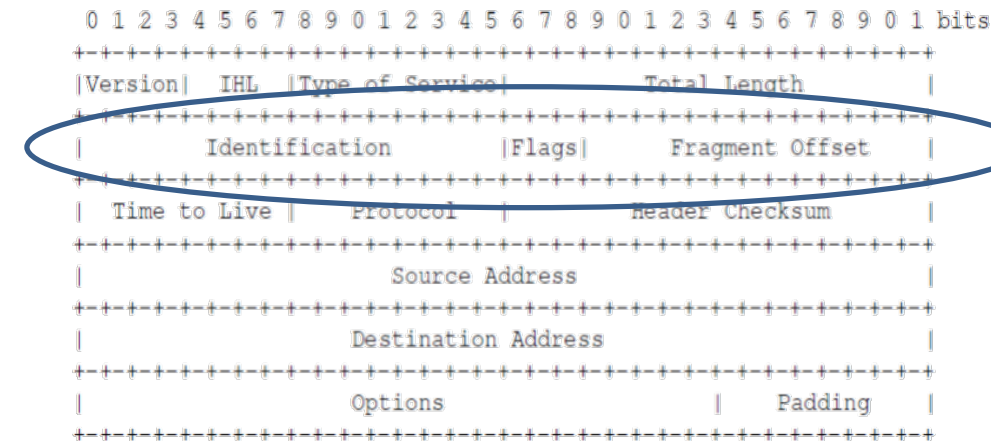
- **Identification:** The IP layer of the source host (the one generating the datagrams) assigns to this field the value of a counter that increases for each new datagram. This number allows to identify fragments of a single message.



Internet Protocol: Fragmentation



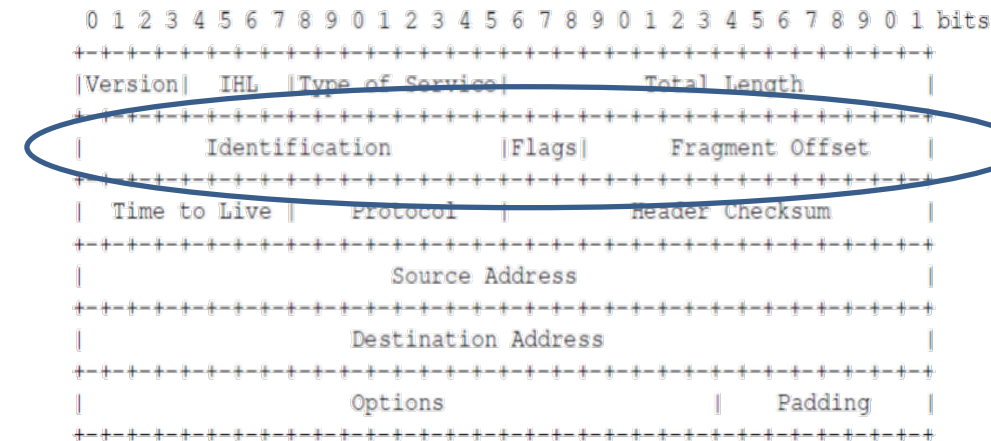
- **Flags (0DM):** the first bit is unused, the rest are:
 - **D: Don't fragment.** When it is set to '1', the datagram cannot be fragmented. If a router needs to fragment, it will discard the datagram and send back an ICMP error message.
 - **M: More fragments.** If set to '1', the flag means that more fragments are coming. All fragments carry this flag to '1' except for the last.



Internet Protocol: Fragmentation



- **Offset:** the position of the first byte of the fragment in the original datagram (it is '0' in the first fragment). It counts 8-byte units (datagrams can be as large as $2^{16}-1$ with 13 bits of offset).



Internet Protocol: Fragmentation

*Let us suppose that an application generates a UDP datagram of 1980 bytes that needs to be sent through an Ethernet network. The IP layer will build a datagram of 2000 bytes: 20 bytes for the IP header + 1980 bytes for the payload. When the IP layer sees that the datagram needs to be routed through an interface with MTU = 1500 bytes, it will need to fragment it. The maximum payload of the fragments **(in 8-byte units)** will be:*

$$\text{Fragment size} = \left\lfloor \frac{1500 - 20}{8} \right\rfloor = 185 \text{ units}$$

Internet Protocol: Fragmentation

Which means, $185 \times 8 = 1.480$ bytes. Two fragments will be thus generated: the first fragment will carry the first 1480 bytes of payload of the original datagram. The second fragment will carry the 500 remaining bytes. When fragmented, the header of the original datagram will be copied and changes in the following fields will be made as follows:

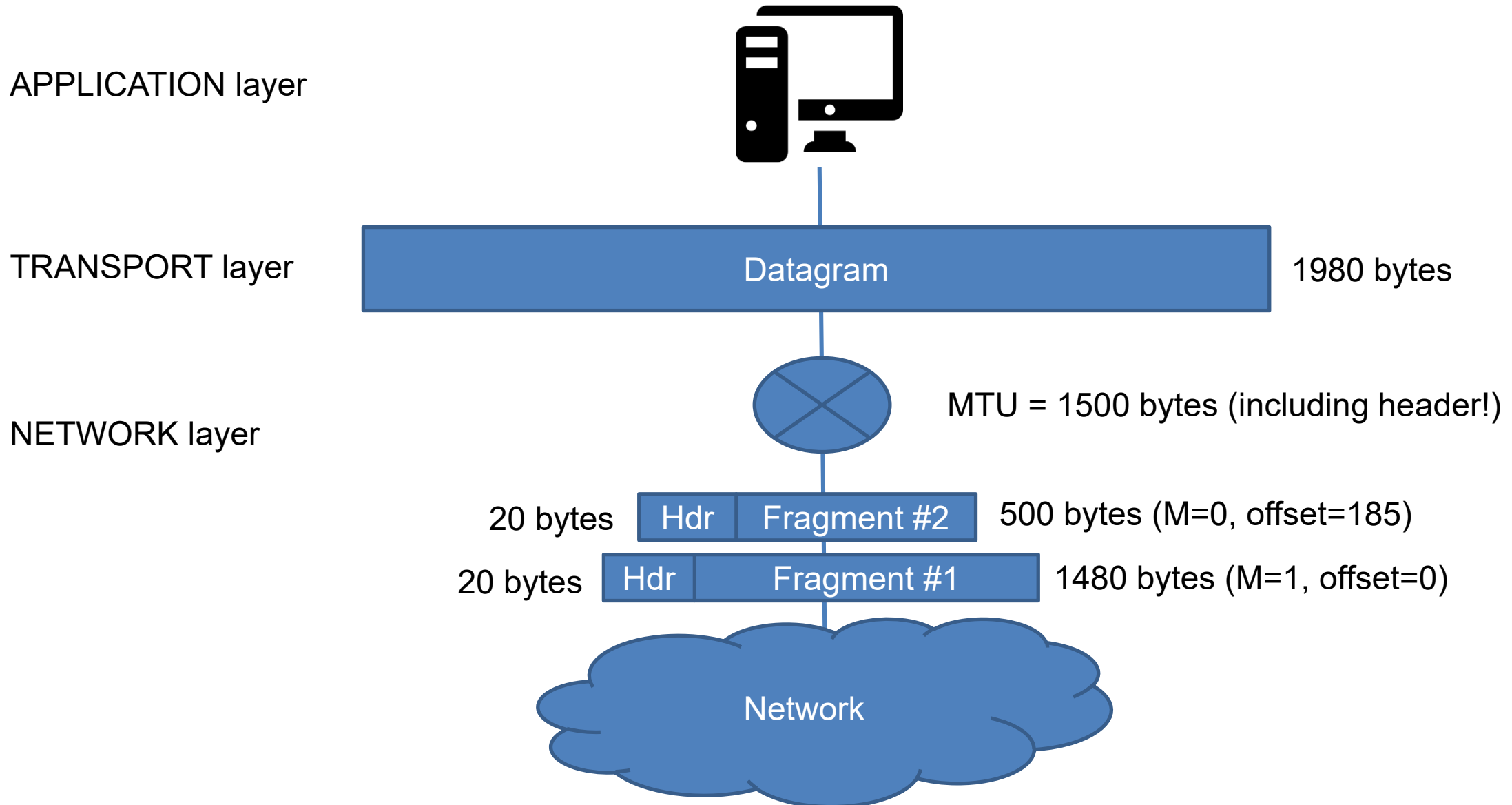
- 1st fragment: offset = 0, M = 1*
- 2nd fragment: offset = 185, M = 0*

Besides the fields above, the IP layer will also need to change the total length field and the checksum of each fragment.

Internet Protocol: Fragmentation



CITM



Fragmentation - Exercise



CITM

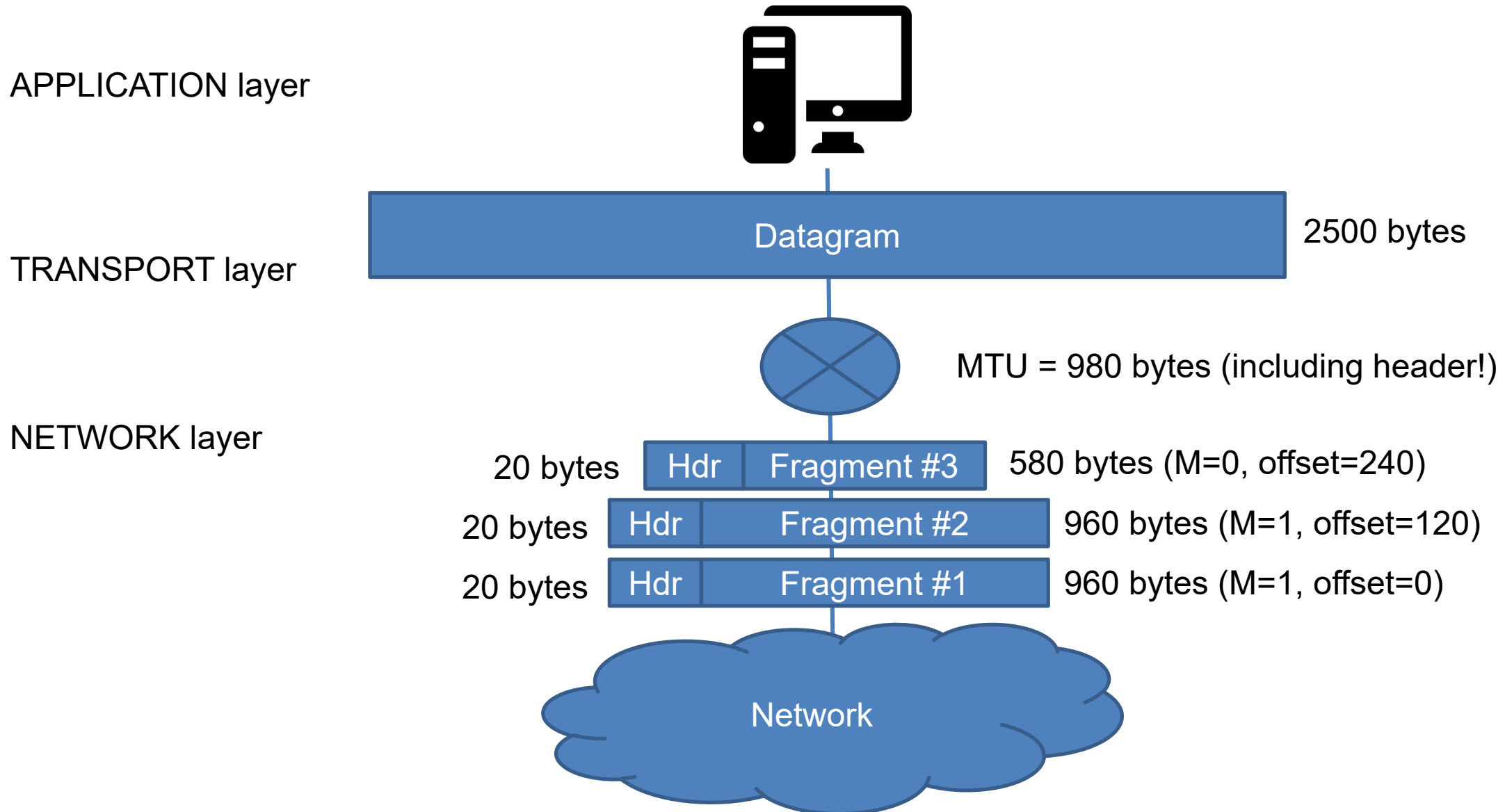
Indicate the values of fields M and offset of the IP datagrams associated to the transfer of a message of 2500 bytes in a network of MTU 980 bytes.



Internet Protocol: Fragmentation



CITM





UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

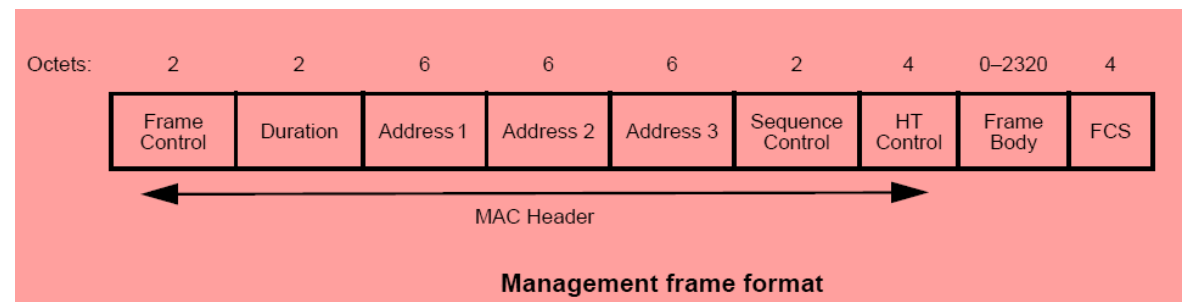
Data Link Layer: Frames, checksums, and knowing when to talk

Data Link Layer

The game

7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Protocols for point to point communication. Managing errors from the physical layer and acknowledging receptions (ACK). Medium access control, flow control.
- **Scope:** two machines connected by a wire
- **Data unit:** frame
- **Example:** 802.11 (WiFi)



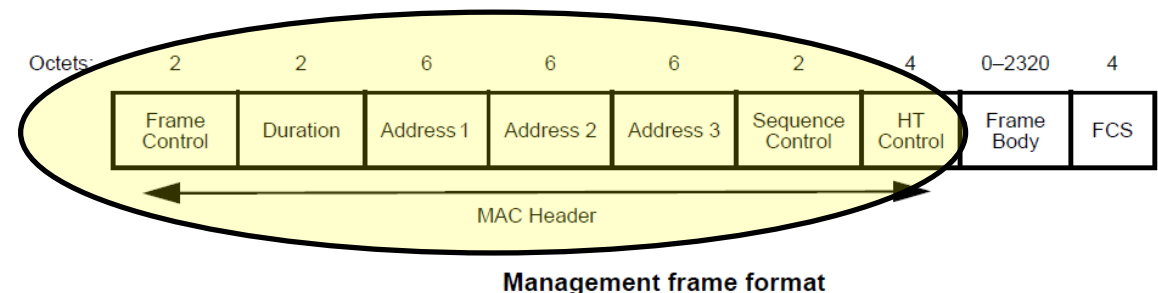
The wire

Data Link Layer

The game

7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Protocols for point to point communication. Managing errors from the physical layer and acknowledging receptions (ACK).
Medium access control, flow control.
- **Scope:** two machines connected by a wire
- **Data unit:** frame
- **Example:** 802.11 (WiFi)



The wire

- Channels may be shared among multiple nodes (wireless?)
- Two transmissions through the same channel at the same time will “collide” and not be understood by the receiver.
- There is a need to manage access to a shared channel in way that maximizes its use. Alternatives:
 1. **Round Robin:** Requests are ordered following a circular order
 2. **Reservation:** Requesting nodes are given an exclusive “air time”
 3. **Contention:** Nodes fight for access using control packets or simply transmitting the data.

Example: CSMA/CA



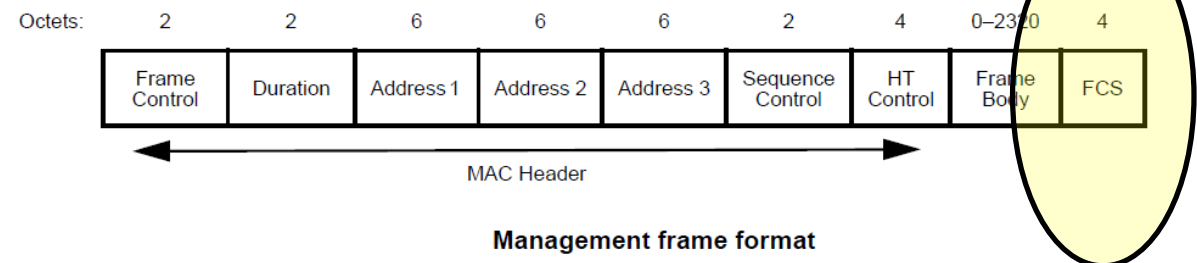
- Carrier Sense Multiple Access with Collision Avoidance
- Contention-based protocol used in 802.11 (WiFi)
- Pseudo-algorithm of a variant:
 - Sense the channel (carrier)
 - If busy, wait until it becomes free and then transmit
 - If idle, transmit
 - After a transmission, wait for an ACKnowledgment
 - If not received, there was a collision. Random backoff and try again.
 - If received, then finish.

Data Link Layer

The game

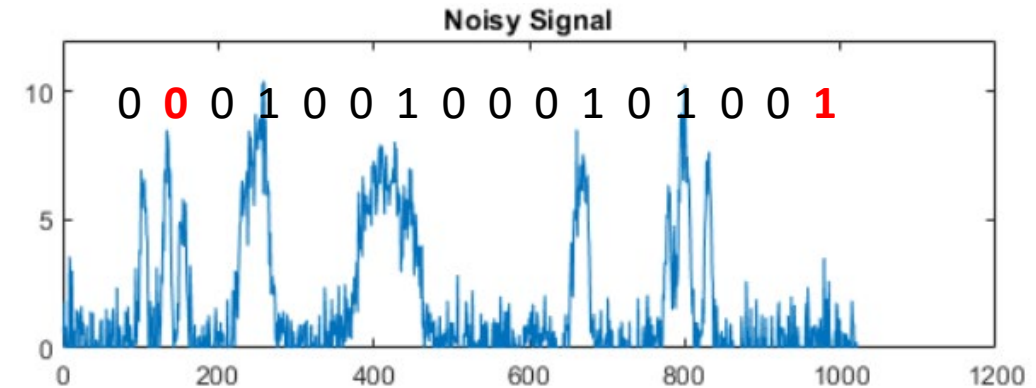
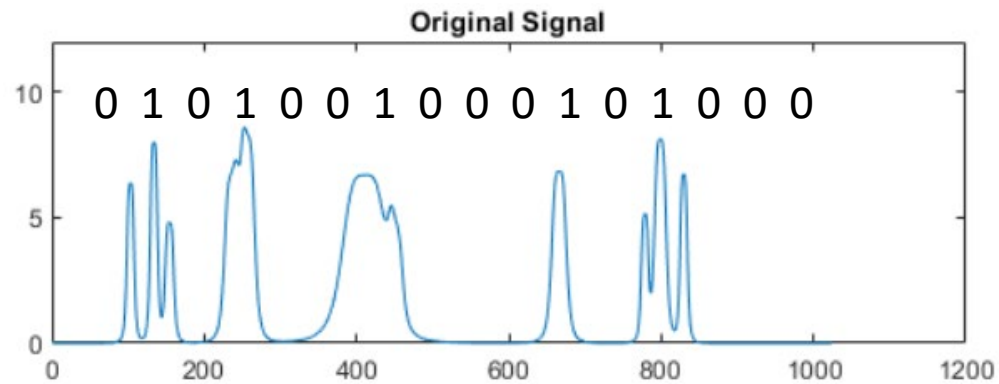
7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

- Protocols for point to point communication. Managing errors from the physical layer and acknowledging receptions (ACK). Medium access control, flow control.
- **Scope:** two machines connected by a wire
- **Data unit:** frame
- **Example:** 802.11 (WiFi)



The wire

Errors in the physical layer



How can we recover from errors happening at the physical layer?

Automatic Repeat ReQuest (ARQ):
Upon detecting an error, the receiver asks the sender to repeat

Forward Error Correction (FEC):
Upon receiving an error, the receiver attempts to correct it himself

HARQ

Codes to detect and correct errors

1. The source makes a given operation with the bits to be transmitted and stores the result at the header of the message.
2. The receiver makes the operation again and compares the result with the value contained in the header.

If the operation reported by the source and the one calculated in reception do not match, then we can say that its content has been manipulated or that have been errors in the transmission. **This will typically not be a 100% robust verification, it depends on the code.**

Examples of codes

- Parity check codes
- The letter of your DNI
- Turbo codes
- Reed-Solomon codes
- Hamming codes
- Cyclic Redundancy Checks (CRC)

Ask me about this!

RESTO	0	1	2	3	4	5	6	7	8	9	10	11
LETRA	T	R	W	A	G	M	Y	F	P	D	X	B

RESTO	12	13	14	15	16	17	18	19	20	21	22
LETRA	N	J	Z	S	Q	V	H	L	C	K	E

- *CRC is used daily*
 - Your hard drive probably uses CRC32 to check errors.
 - Ethernet protocol (link level) uses 32-bit checksum
 - IP protocol uses checksum in the header to verify that the addresses are correct
 - TCP protocol uses checksum protecting the data and the IP addresses as well.

Exercise



CITM

- For you to practice and tinker

LibCRC: CRC Open
Source Library for C.
© MIT.

<https://www.libcrc.org/>



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

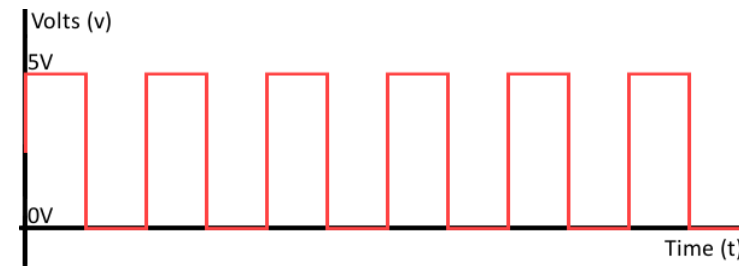
Physical Layer: Pins and modulations

The OSI Layers

The game

7	Application
6	Presentation
5	Session
4	Transport
3	Network
2	Data Link
1	Physical

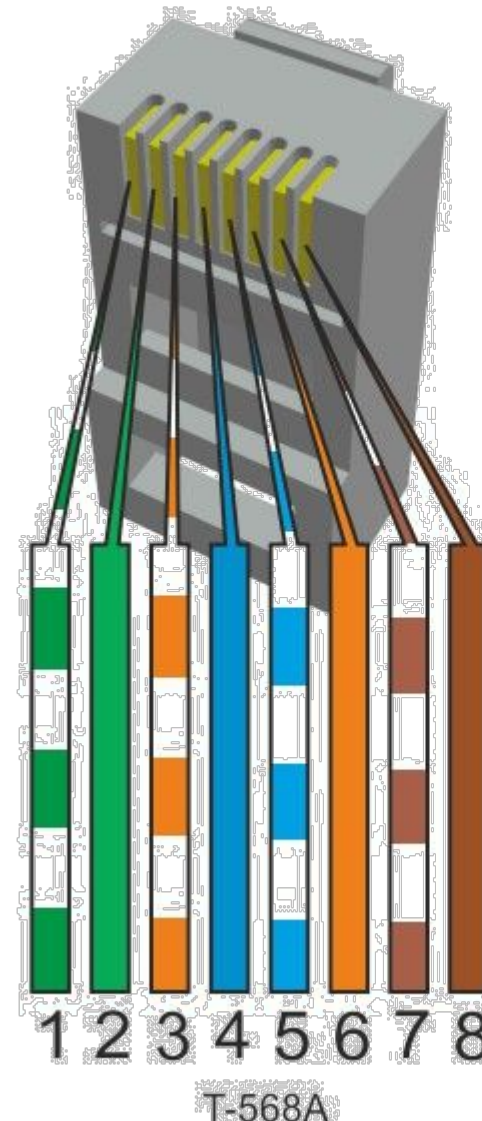
- Protocols that interface with the communications medium. Representation of raw bits and signal specifications
- **Scope:** within the comm. medium
- **Data unit:** symbol
- **Example:** X.21



The wire

An example

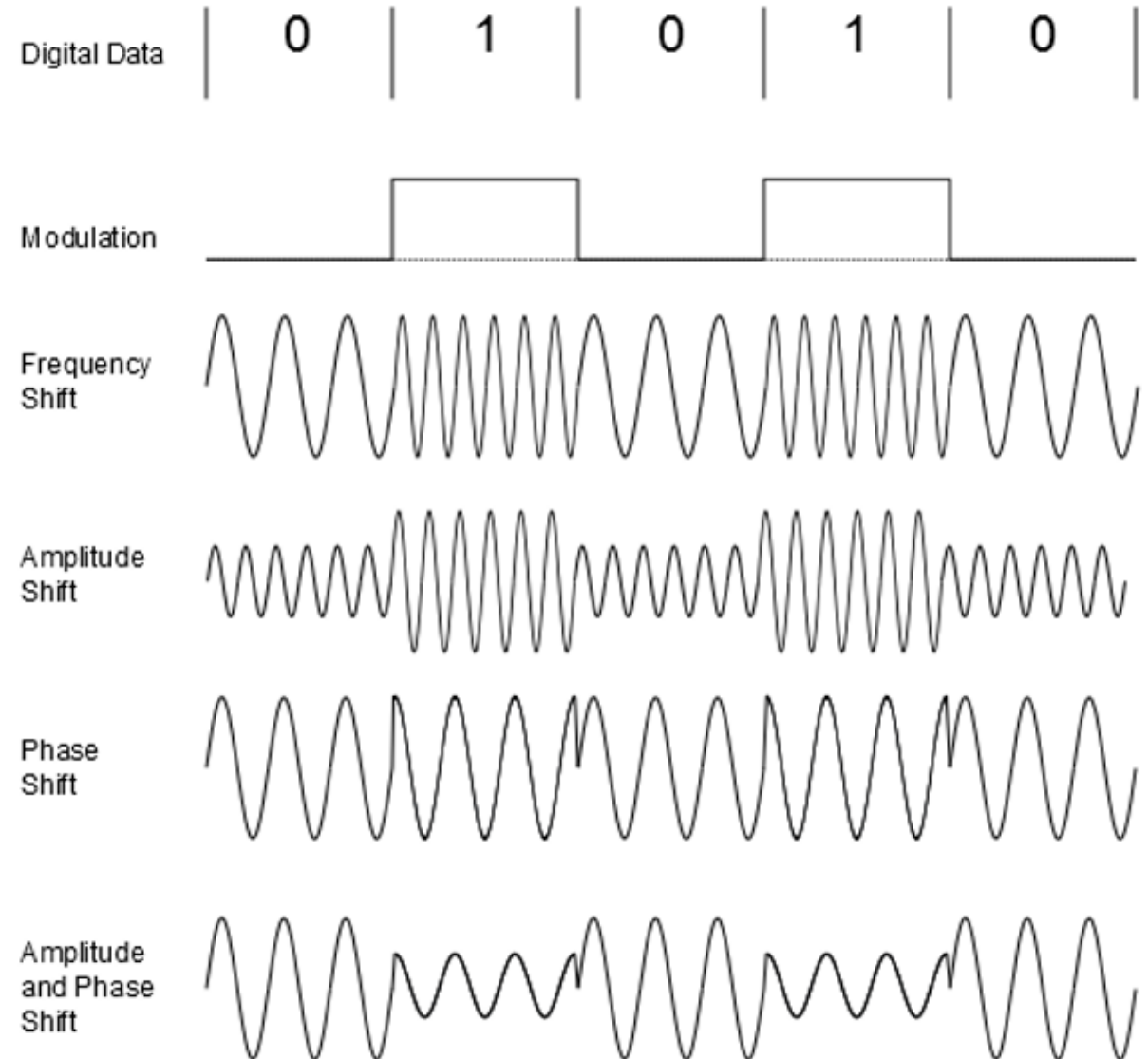
- Ethernet PHY:
 - How many symbols can I send at the same time? (pins 1 & 2)
 - Can I receive at the same time that I sent? (pins 3 and 6)
 - **How do we represent the symbols in each pin? (PAM-3)**
 - How fast we send the symbols? (125 times per second)



Pin	Description	10base-T	100Base-T	1000Base-T
1	Transmit Data+ or BiDirectional	TX+	TX+	BI_DA+
2	Transmit Data- or BiDirectional	TX-	TX-	BI_DA-
3	Receive Data+ or BiDirectional	RX+	RX+	BI_DB+
4	Not connected or BiDirectional	n/c	n/c	BI_DC+
5	Not connected or BiDirectional	n/c	n/c	BI_DC-
6	Receive Data- or BiDirectional	RX-	RX-	BI_DB-
7	Not connected or BiDirectional	n/c	n/c	BI_DD+
8	Not connected or BiDirectional	n/c	n/c	BI_DD-

Modulation

- Our computer works with digital (discrete) data.
- Cables and air work with analog (continuous) data.
- The physical layer determines how to convert and transmit it through the communications channel.





UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

The Protocol Stack

2.2 Lower layers: Network, link, physical

Sergi Abadal (abadal@ac.upc.edu)

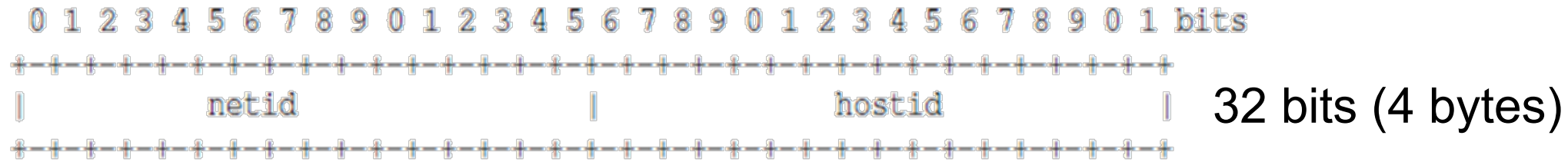


UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH

Centre de la Imatge i la Tecnologia Multimèdia

Extra material

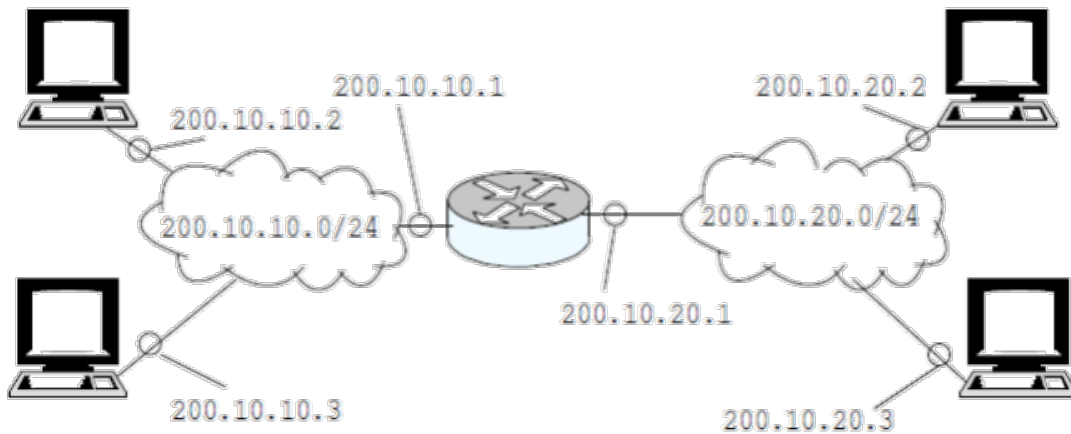
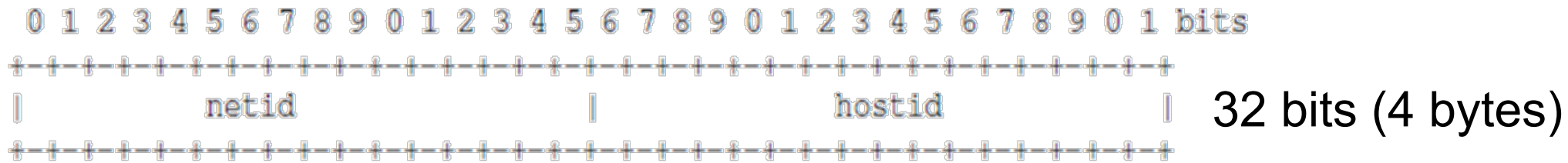
Internet Protocol: IPv4 addresses



Addresses generally come with a “mask” that decides the division of bits between netid and hostid.

Class	Range	Mask	Used for
A	0.0.0.0 to 127.255.255.255	255.0.0.0 (/8)	Very large networks
B	128.0.0.0 to 191.255.255.255	255.255.0.0 (/16)	Medium networks
C	192.0.0.0 to 223.255.255.255	255.255.255.0 (/24)	Small networks
D	224.0.0.0 to 239.255.255.255		Multicast
E	240.0.0.0 to 247.255.255.255		Experimental

Internet Protocol: Public IP addresses



- **The addresses need to be unique through the entire Internet.**
- To achieve this, the Internet Assigned Numbers Authority (IANA) assigns address blocks to the Regional Internet Registries (RIR):
 - RIPE: Europa (<http://www.ripe.net>)
 - ARIN: USA (<http://www.arin.net>)
 - APNIC: ASIA (<http://www.apnic.net>)
 - LACNIC: Latin America (<http://www.lacnic.net>)

Cyclic Redundancy Check (CRC)



- CRC is a more complex version of a parity check where the position of the bit is used in the calculation.
- Unlike in a checksum, this allows to detect permutations.
- CRC defines two polynomials $P(x)$ and $G(x)$
 - $P(x)$: represents the data. “1” in position ‘n’ is represented by x^n and each “0” in position ‘m’ implies the absence of x^m
 - Example: 01101001 $\gg P(x) = x^6 + x^5 + x^3 + x^0$
 - $G(x)$: generator polynomial, is used to encode and decode. It depends on the actual algorithm:
 - Example: CRC-32 $\gg G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$

Encoding and decoding a CRC

SOURCE

- $P(x)$ is multiplied by the 'r' degree of $G(x)$ to obtain $M(x)$. This means that r 0's are added to the original message.
- $M(x)/G(x)$ [**XOR DIVISION**] is calculated to obtain the remainder $R(x)$
- The transmitted message will contain $P(x)$ and 'r' bits of $R(x)$

DESTINATION

- Divide the received message by $G(x)$
- If the remainder of the division is not equal to zero, then it means that the message has been corrupted or manipulated during transmission.

Encoding and decoding a CRC

SOURCE

- $P(x)$ is multiplied by the 'r' degree of $G(x)$ to obtain $M(x)$. This means that r 0's are added to the original message.
- $M(x)/G(x)$ **[XOR DIVISION]** is calculated to obtain the remainder $R(x)$
- The transmitted message will contain $P(x)$ and 'r' bits of $R(x)$

DESTINATION

- Divide the received message by $G(x)$
- If the remainder of the division is not equal to zero, then it means that the message has been corrupted or manipulated during transmission.

XOR Division



CITM

Division with special rules:

- Dividend always starts by 1
- Bring down as many digits as needed so that the dividend and divisor have the same number of digits.
- Subtraction is XOR
 - 1 if digits are different
 - 0 if digits are equal

The diagram illustrates the XOR division process. It shows a long division setup where the divisor 1001 is subtracted from the dividend 10011011010 using XOR. The process is shown in three steps, with the current state of the dividend and the divisor aligned for XOR. Blue circles and arrows highlight the steps: the first circle highlights the initial state, the second circle highlights the first XOR operation, and the third circle highlights the second XOR operation. The final result is 0000.

$$\begin{array}{r} 1001 \overline{) 10011011010} \\ \underline{1001} \\ 00001011 \\ \underline{1001} \\ 001001 \\ \underline{1001} \\ 00000 \end{array}$$

CRC: Example



- Generate the message to transmit after adding the CRC to the word “1010” given $G(x) = x^2 + x + 1$

$$P(x) = x^3 + x$$

$$M(x) = P(x) \cdot x^2 \rightarrow M(x) = x^5 + x^3$$

$$R(x) = M(x) \bmod G(x) \rightarrow R(x) = x$$

Message to transmit: 101010

Exercise



CITM

- Verify whether the message “10011011010”, which contains an 8-bit word and a CRC code of 3 bits, generated using polynomial $G(x)=x^3 + 1$, has been manipulated or corrupted.



CRC: Example from transmitter side

- Verify whether the message “10011011010”, which contains an 8-bit word and a CRC code of 3 bits, generated using polynomial $G(x)=x^3 + 1$, has been manipulated or corrupted.
 - Word to transmit (8 bits): 10011011
 - Generator polynomial: $G(x) = x^3 + 1 \rightarrow 1001$
 - Degree of the generator (r): 3
 - We add ‘r’ zeroes at the end: 10011011**000**
 - We divide $x^3 \cdot M(x)$ by $G(x)$
...this is: 10011011000 div 1001
 - We check if the remainder matches the three last bits of the message “010”

CRC: Example



CITM

10011011000	1001	10011011000	1001	10011011000	1001
		<u>1001</u>	1	<u>1001</u>	10001
		0000		00001011	
				<u>1001</u>	
				0010	
10011011000	1001	10011011000	1001	10011011000	1001
<u>1001</u>	1000101	<u>1001</u>	10001010	<u>1001</u>	10001010
00001011		00001011		00001011	
<u>1001</u>		<u>1001</u>		<u>1001</u>	
001000		001000		001000	
<u>1001</u>		<u>1001</u>		<u>1001</u>	
0001		00010		000 10	

The message has not
been manipulated

CRC: Example from receiver side

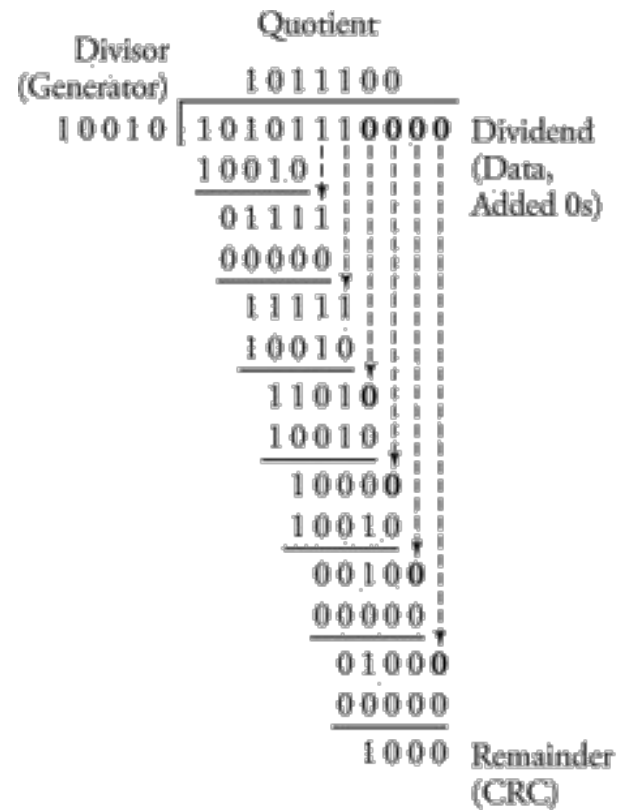
- Verify whether the message “10011011010”, which contains an 8-bit word and a CRC code of 3 bits, generated using polynomial $G(x)=x^3 + 1$, has been manipulated or corrupted.
 - Word to transmit (8 bits): 10011011
 - Generator polynomial: $G(x) = x^3 + 1 \rightarrow 1001$
 - Degree of the generator (r): 3
 - We divide the entire message by the code of the generator
 - Check if the remainder is 0.

$$\begin{array}{r} 10011011010 \quad | \quad 1001 \\ \underline{1001} \\ 00001011 \\ \underline{1001} \\ 001001 \\ \underline{1001} \\ 00000 \end{array}$$

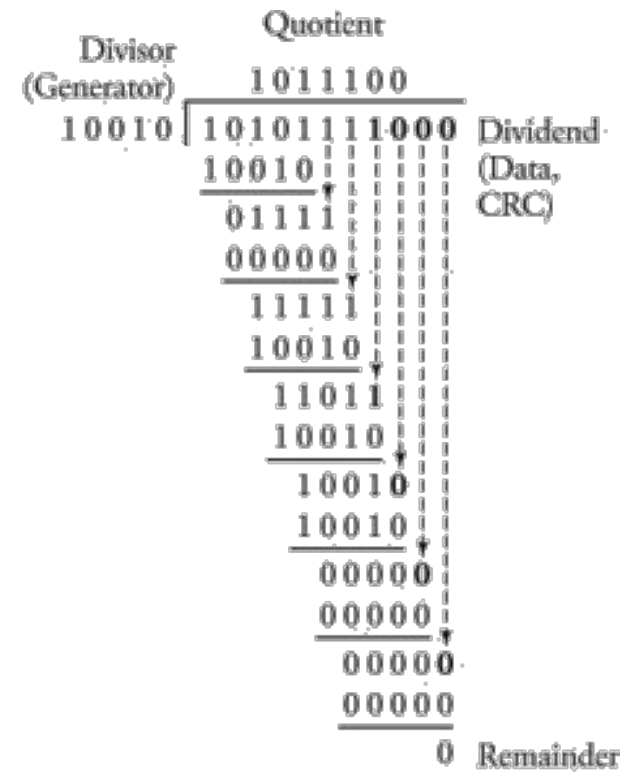
CRC: Another example



- Message “10101111000” with a 7-bit word and a 4-bit CRC code generated with $G(x)=x^4+x$



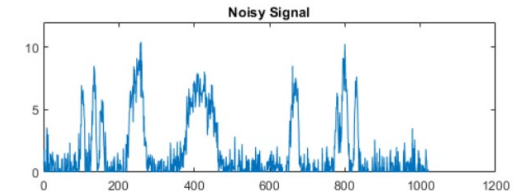
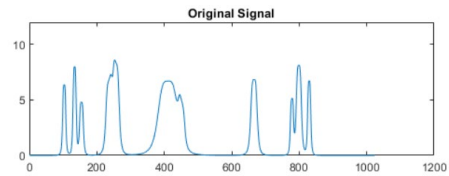
(a) Transmitter



(b) Receiver

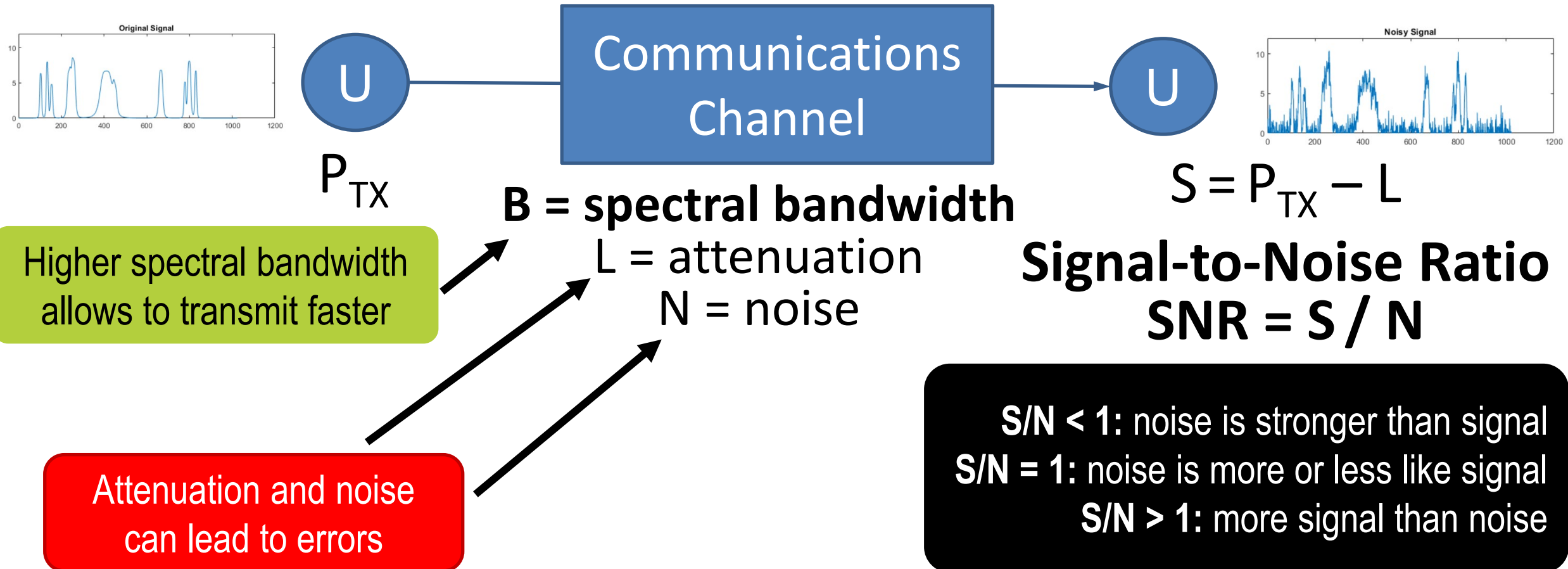
- *I'm not making this up – CRC is used daily*
 - Your hard drive probably uses CRC32 to check errors.
 - Ethernet protocol (link level) uses 32-bit checksum
 - IP protocol uses checksum in the header to verify that the addresses are correct
 - TCP protocol uses checksum protecting the data and the IP addresses as well.

The communications channel



B = spectral bandwidth
L = attenuation
N = noise

The communications channel

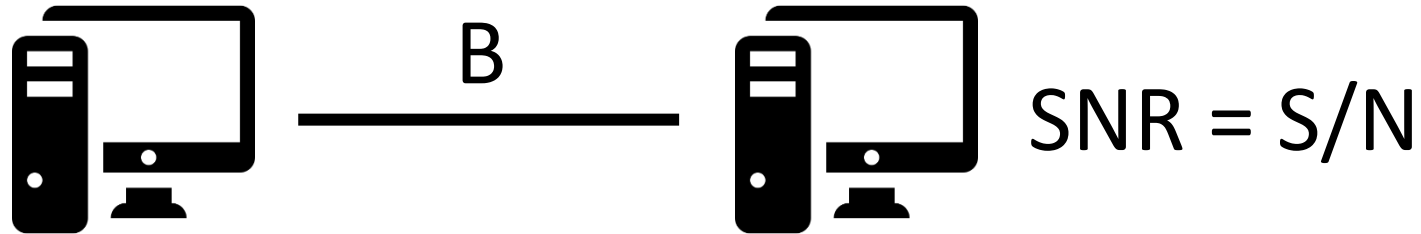


*Shannon's Theorem tells us the capacity (**maximum speed**) of any communications channel based on its spectral bandwidth and signal-to-noise ratio.*

Shannon's Theorem



CITM



$$C = B \cdot \log_2 \left(1 + \frac{S}{N} \right)$$

C: channel capacity (bps = b/s)

B: spectral bandwidth (Hertz)

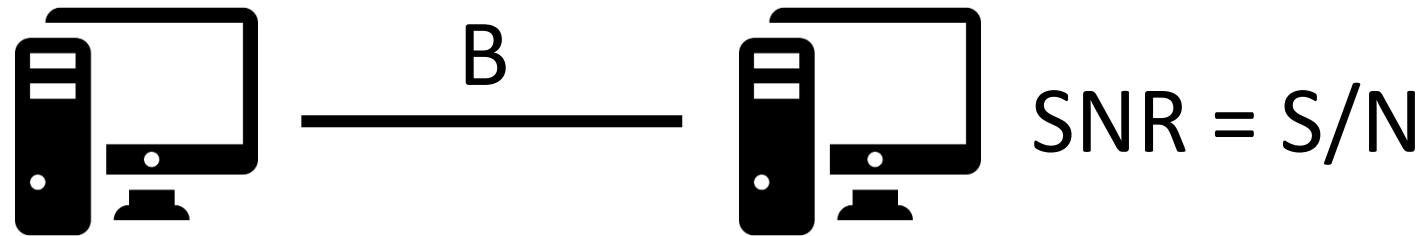
S: signal power (W, mW, nW)

N: noise power (same units than S)

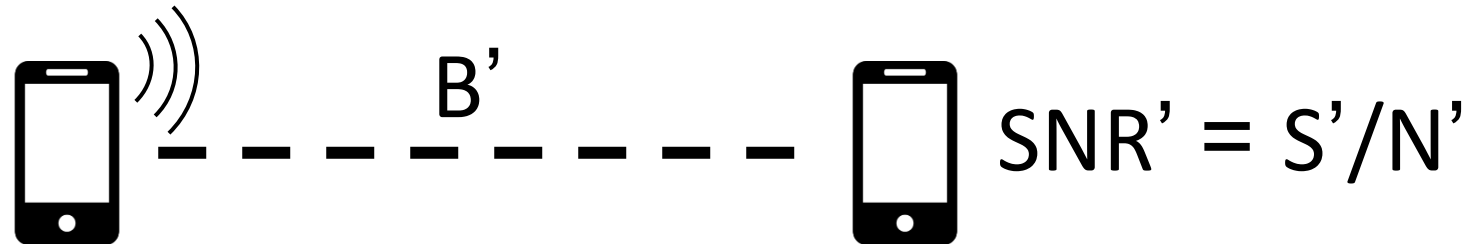
Physical Layer: a transmission



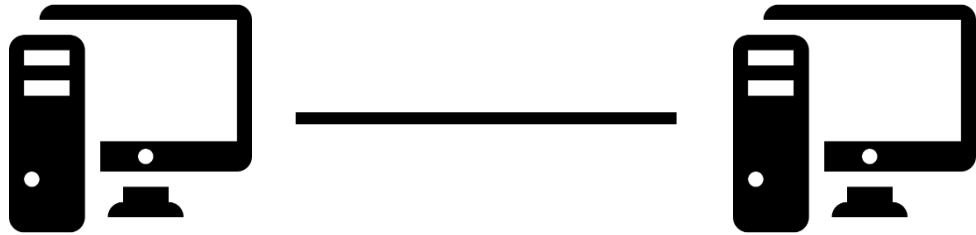
CITM



Which channel is faster?



Shannon's Theorem

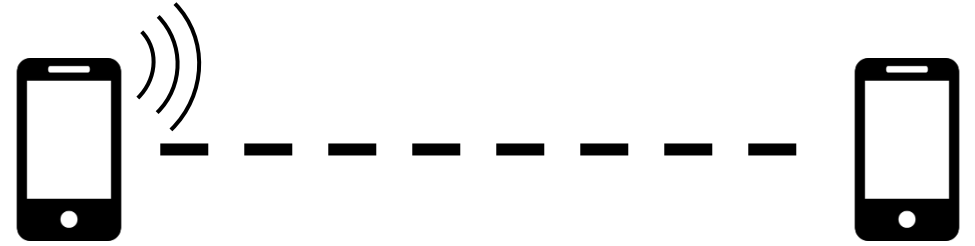


$$B = 100 \text{ MHz}$$

$$S = 10 \text{ mW}$$

$$N = 0.001 \text{ mW}$$

$$C = 100 \cdot 10^6 \cdot \log_2 \left(1 + \frac{10}{0.001} \right) = \\ = 1.32 \text{ Gb/s}$$



$$B = 40 \text{ MHz}$$

$$S = 1 \text{ mW}$$

$$N = 0.01 \text{ mW}$$

$$C = 40 \cdot 10^6 \cdot \log_2 \left(1 + \frac{1}{0.01} \right) = \\ = 266.33 \text{ Mb/s}$$

Exercises



CITM

- What happens if we multiply the bandwidth by 10X?
- What happens if we multiply the S/N by 10X?

